

Estructuras de Datos

Clase 13 – Árboles Generales

(primera parte)

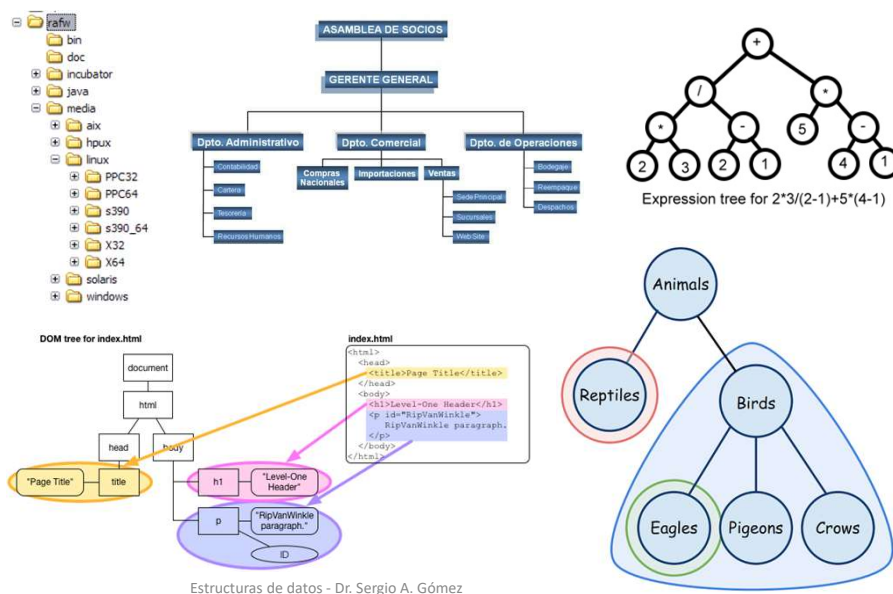


Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



Departamento de Ciencias e Ingeniería de la Computación
 Universidad Nacional del Sur
 Bahía Blanca, Argentina

Estructuras de datos jerárquicas: motivaciones

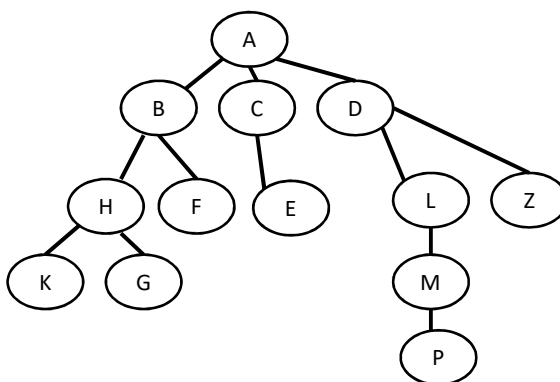


El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Preliminares

- Un árbol es un TDA que almacena los elementos llamados *nodos* jerárquicamente
- Con la excepción del nodo *raíz*, cada nodo en un árbol tiene un nodo padre y cero o más nodos hijos.
- La raíz se dibuja arriba.

El nodo con rótulo A es la raíz.
El nodo con rótulo B es el padre los nodos con rótulos H y F.
El nodo con rótulo F es el hijo del nodo con rótulo B.



Estructuras de datos - Dr. Sergio A. Gómez

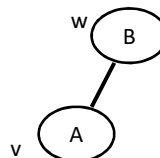
3

Definición formal de árbol

Un árbol T se define como un conjunto de nodos almacenando elementos tales que los nodos tienen una relación padre-hijo, que satisface:

- Si T es no vacío, tiene un nodo especial, llamado la raíz de T , que no tiene padre.
- Cada nodo v de T diferente de la raíz tiene un único nodo padre w
- Cada nodo v con padre w es un hijo de w .

A es el rótulo de v .
B es el rótulo de w .
Cuando no haya confusión, nos referiremos al nodo con su rótulo.



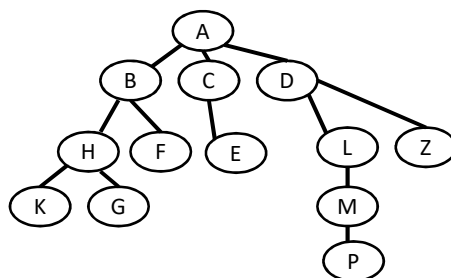
Estructuras de datos - Dr. Sergio A. Gómez

4

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Relaciones entre nodos

- Nodos Hermanos: Dos nodos con el mismo padre se llaman hermanos. Ej. B y D son hermanos, L y Z lo son, H y E no son hermanos (son primos).
- Nodos externos u hojas: Un nodo v es externo si no tiene hijos. Ej: F, K, G, E, P y Z son hojas.
- Nodo interno: Un nodo v es interno si tiene uno o más hijos. Ej: A, B, C, D, H, L y M son nodos internos.

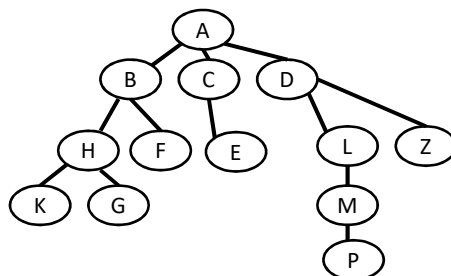


Estructuras de datos - Dr. Sergio A. Gómez

5

Más relaciones entre nodos

- Ancestro(u,v): Un nodo u es ancestro de un nodo v si $u=v$ o u es un ancestro del padre de v .
Ej: ancestro(C,C), ancestro(A,E), ancestro(D,P)
- Ancestro propio(u,v): u es ancestro propio de v si u es ancestro de v y $u \neq v$.
Ej: ancestropropio(A,E), ancestropropio(D, P), ancestropropio(D,L)

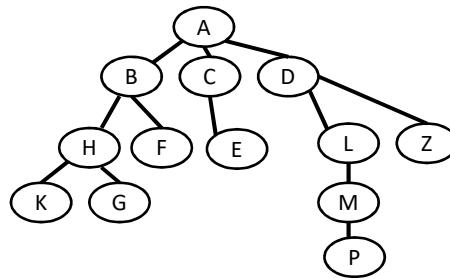


Estructuras de datos - Dr. Sergio A. Gómez

6

Y más relaciones entre nodos

- Descendiente(u,v): Un nodo u es descendiente de un nodo v si v es un ancestro de u.
- Ej: descendiente(C,C), descendiente(E,A), descendiente(P,D)
- Descendiente propio(u,v): u es descendiente propio de v si u es descendiente de v y $u \neq v$.
- Ej: descendientepropio(E,A).

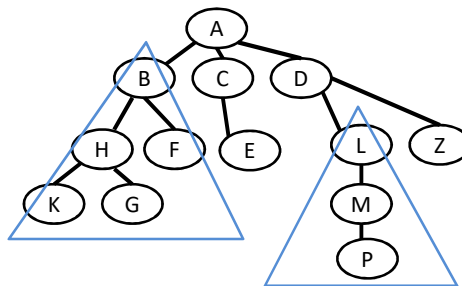


Estructuras de datos - Dr. Sergio A. Gómez

7

Y más relaciones entre nodos

- Subárbol: El subárbol de T con raíz en el nodo v es el árbol consistiendo de todos los descendientes de v.
- Ej: Mostramos dos subárboles con raíces B y L respectivamente.



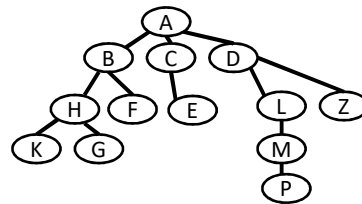
Estructuras de datos - Dr. Sergio A. Gómez

8

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Arcos y caminos en árboles

- Arco dirigido: Un arco de un árbol T es un par de nodos (u,v) tales que u es el padre de v .
- Ej: (D,L) es un arco y (D,Z) es otro arco.
- Arco no dirigido: Un arco es un par de nodos (u,v) o (v,u) tales que u es padre de v .
- Ej: (D,Z) y (Z,D) son arcos no dirigidos.
- Camino: Un camino de T es una secuencia de nodos tales que cualquier par de nodos consecutivos en la secuencia forman un arco.
- Ej: A, B, F es un camino
- Ej: F, B, A, D, L, M es otro camino.



Estructuras de datos - Dr. Sergio A. Gómez

9

Arcos y caminos en árboles

- Ejemplo: La relación de herencia de clases en Java forman un árbol.
- La raíz, `java.lang.Object` es un ancestro de todas las clases.
- Cada clase C es un descendiente de `Object` y C es la raíz del subárbol formado por todas las clases descendientes de C .

Estructuras de datos - Dr. Sergio A. Gómez

10

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Árboles ordenados

- Un árbol se dice ordenado si existe un orden lineal para los hijos de cada nodo,
- Es decir, se puede identificar el primer hijo, el segundo hijo y así sucesivamente
- Tal orden se visualiza de izquierda a derecha de acuerdo a tal ordenamiento.
- Ejemplo:
 - Los componentes de un libro:
 - El libro es la raíz,
 - parte, capítulo, sección, subsección, subsubsección, son los nodos internos
 - Los párrafos, figuras, tablas son las hojas.

Estructuras de datos - Dr. Sergio A. Gómez

11

ADT Árbol

- Position:
 - `element()`: retorna el objeto almacenado en esta posición
- Tree: Métodos de acceso (reciben y retornan posiciones)
 - `root()`: Retorna la raíz del árbol, error si el árbol está vacío
 - `parent(v)`: Retorna el padre de `v`, error si `v` es la raíz
 - `children(v)`: Retorna una colección *iterable* conteniendo los hijos del nodo `v`

Nota: Si el árbol es ordenado, `children` los mantiene en orden.
Si `v` es una hoja `children(v)` está vacía.

Estructuras de datos - Dr. Sergio A. Gómez

12

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

ADT Árbol

Ejemplos:

$T.root() \rightarrow p_A$

$T.parent(p_F) \rightarrow p_B$

$T.parent(p_M) \rightarrow p_L$

$T.parent(p_A) \rightarrow \text{error}$

$T.children(p_B) \rightarrow [p_H, p_F]$

$T.children(p_A) \rightarrow [p_B, p_C, p_D]$

$T.children(p_E) \rightarrow []$

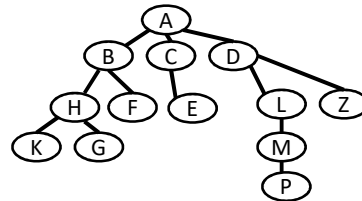
$T.children(p_G) \rightarrow []$

$p_G.element() \rightarrow 'G'$

$p_A.element() \rightarrow 'A'$

$T.root().element() \rightarrow 'A'$

$T.children(T.root()).iterator().next().element() \rightarrow 'B'$



T

Estructuras de datos - Dr. Sergio A. Gómez

13

ADT Arbol

- Tree: Métodos de consulta
 - $isInternal(v)$: Testea si v es un nodo interno
 - $isExternal(v)$: Testea si v es una hoja
 - $isRoot(v)$: Testea si v es la raíz

Ejemplos:

$T.isInternal(T.root()) \rightarrow \text{true}$

$T.isInternal(p_G) \rightarrow \text{false}$

$T.isInternal(p_H) \rightarrow \text{true}$

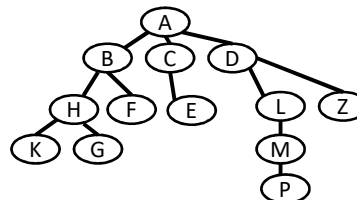
$T.isExternal(T.root()) \rightarrow \text{false}$

$T.isExternal(p_G) \rightarrow \text{true}$

$T.isExternal(p_H) \rightarrow \text{false}$

$T.isRoot(p_G) \rightarrow \text{false}$

$T.isRoot(T.root()) \rightarrow \text{true}$



T

Estructuras de datos - Dr. Sergio A. Gómez

14

ADT Arbol

- Tree: Métodos genéricos
 - size(): Retorna el número de nodos del árbol
 - isEmpty(): Testea si el árbol tiene o no nodos
 - iterator(): Retorna un iterador con los elementos ubicados en los nodos del árbol
 - positions(): Retorna una colección iterable de los nodos del árbol
 - replace(v,e): Reemplaza con e y retorna el elemento ubicado en v.

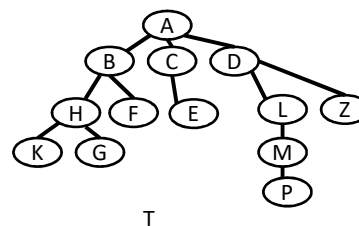
Ejemplos:

T.size() → 13

T.isEmpty() → false

Imprimir rótulos del árbol T:

```
for( char c : T ) { .... } ó:
Iterator<Character> it = T.iterator();
while( it.hasNext() ) { char c = it.next(); ... } ó:
for( Position<Character> p : T.positions() )
{ .... p.element() .... }
```



Estructuras de datos - Dr. Sergio A. Gómez

15

ADT Arbol

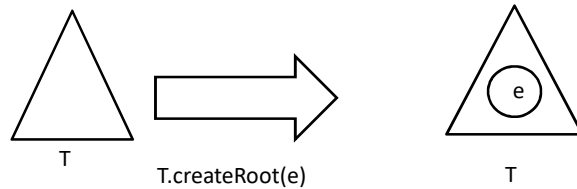
- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - createRoot(e): crea un nodo raíz con rótulo e
 - addFirstChild(p, e): agrega un primer hijo al nodo p con rótulo e
 - addLastChild(p, e): agrega un último hijo al nodo p con rótulo e
 - addBefore(p, rb, e): Agrega un nodo con rótulo e como hijo de un nodo padre p dado. El nuevo nodo se agregará delante de otro nodo hermano rb también dado.
 - addAfter(p, lb, e): Agrega un nodo con rótulo e como hijo de un nodo padre p dado. El nuevo nodo se agregará detrás de otro hermano lb también dado.

Estructuras de datos - Dr. Sergio A. Gómez

16

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - createRoot(e): crea un nodo raíz con rótulo e

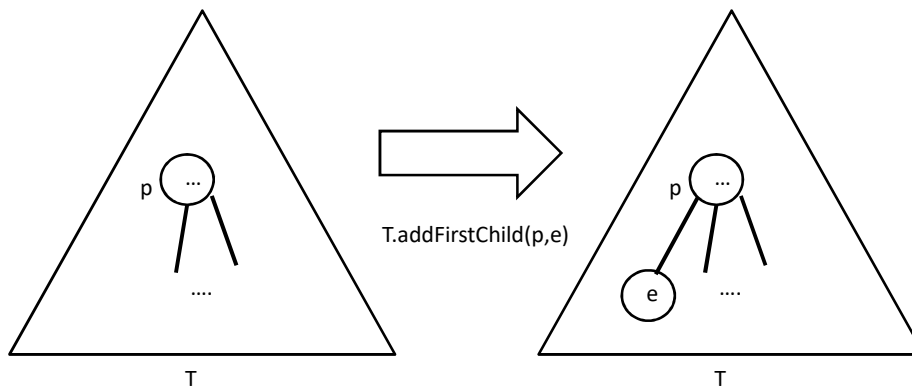


Estructuras de datos - Dr. Sergio A. Gómez

17

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - addFirstChild(p, e): agrega un primer hijo al nodo p con rótulo e



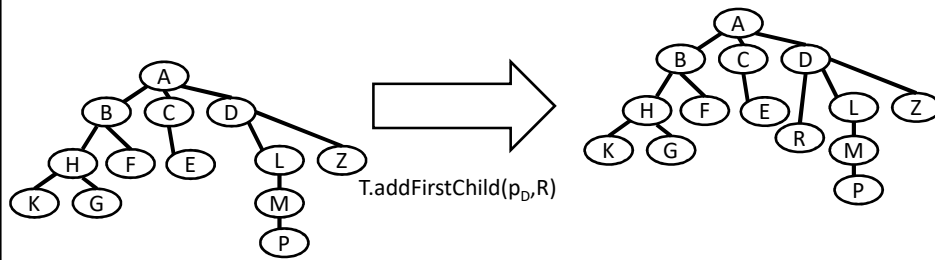
Estructuras de datos - Dr. Sergio A. Gómez

18

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - addFirstChild(p, e): agrega un primer hijo al nodo p con rótulo e

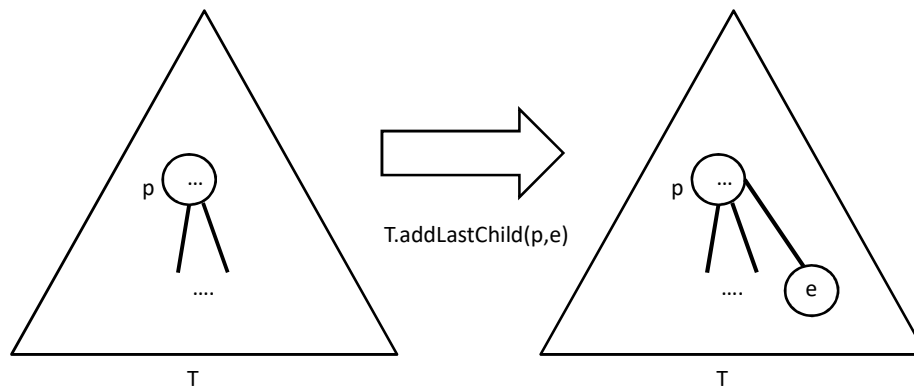


Estructuras de datos - Dr. Sergio A. Gómez

19

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - addLastChild(p, e): agrega un último hijo al nodo p con rótulo e

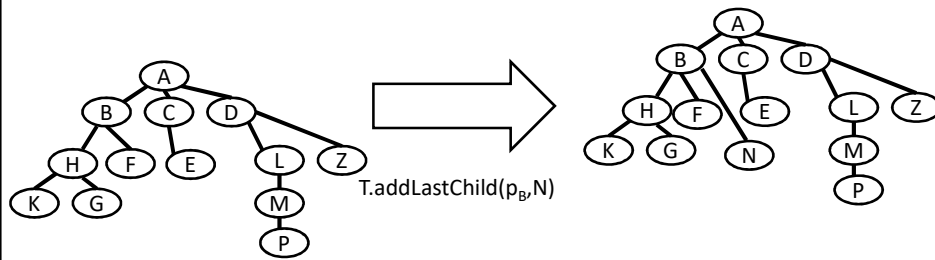


Estructuras de datos - Dr. Sergio A. Gómez

20

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - addLastChild(p , e): agrega un último hijo al nodo p con rótulo e

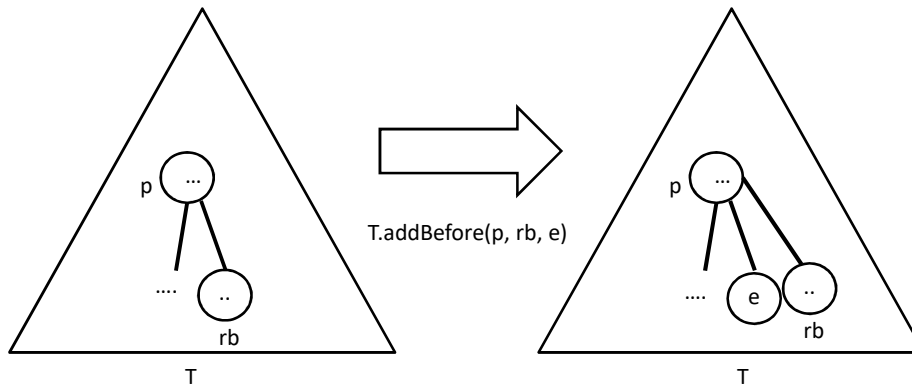


Estructuras de datos - Dr. Sergio A. Gómez

21

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - addBefore(p , rb , e): Agrega un nodo con rótulo e como hijo de un nodo padre p dado. El nuevo nodo se agregará delante de otro nodo hermano rb también dado.

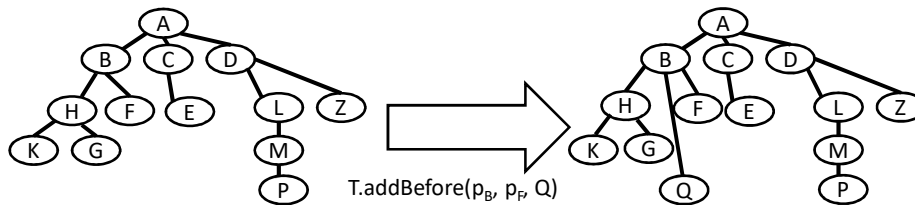


Estructuras de datos - Dr. Sergio A. Gómez

22

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - `addBefore(p, rb, e)`: Agrega un nodo con rótulo e como hijo de un nodo padre p dado. El nuevo nodo se agregará delante de otro nodo hermano rb también dado.

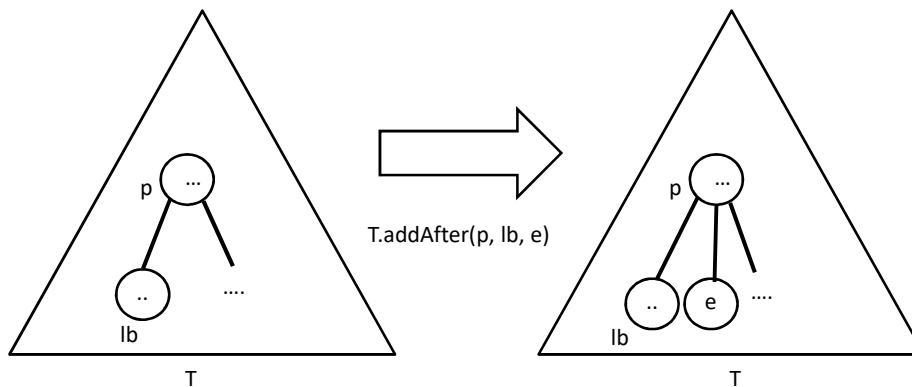


Estructuras de datos - Dr. Sergio A. Gómez

23

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - `addAfter(p, lb, e)`: Agrega un nodo con rótulo e como hijo de un nodo padre p dado. El nuevo nodo se agregará detrás de otro hermano lb también dado.

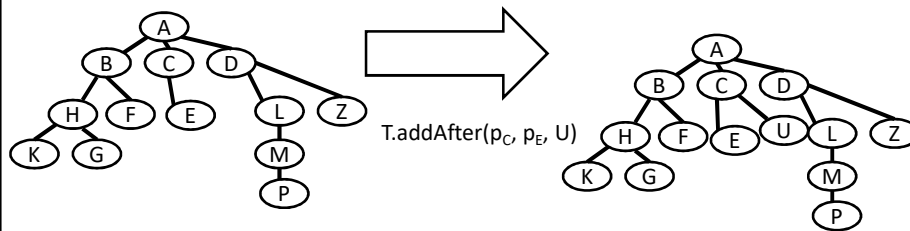


Estructuras de datos - Dr. Sergio A. Gómez

24

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - `addAfter(p, lb, e)`: Agrega un nodo con rótulo e como hijo de un nodo padre p dado. El nuevo nodo se agregará detrás de otro hermano lb también dado.



Estructuras de datos - Dr. Sergio A. Gómez

25

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - `removeExternalNode(p)`: Elimina la hoja p
 - `removeInternalNode(p)`: Elimina el nodo interno p . Los hijos del nodo eliminado lo reemplazan en el mismo orden en el que aparecen. La raíz se puede eliminar si tiene un único hijo.
 - `removeNode(p)`: Elimina el nodo p .

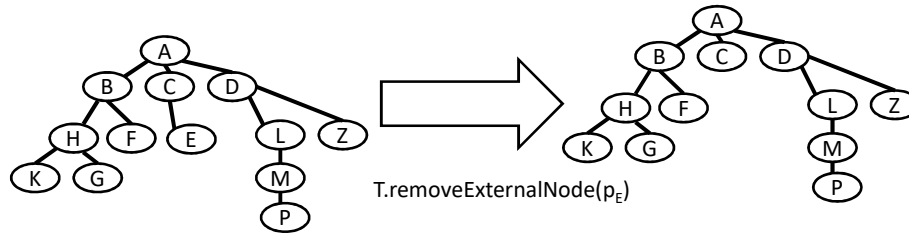
Estructuras de datos - Dr. Sergio A. Gómez

26

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - `removeExternalNode(p)`: Elimina la hoja p

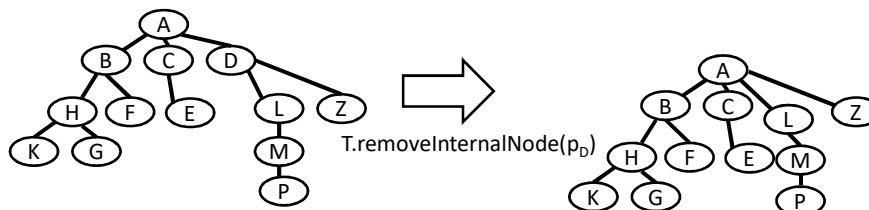


Estructuras de datos - Dr. Sergio A. Gómez

27

ADT Arbol

- Tree: Métodos de modificación (agregados por la cátedra a [GT])
 - `removeInternalNode(p)`: Elimina el nodo interno p . Los hijos del nodo eliminado lo reemplazan en el mismo orden en el que aparecen. La raíz se puede eliminar sólo si tiene un único hijo.



Estructuras de datos - Dr. Sergio A. Gómez

28

```

public interface Tree<E> extends Iterable<E>
{
    public int size();
    public boolean isEmpty();
    public Iterator<E> iterator();
    public Iterable<Position<E>> positions();
    public E replace(Position<E> v, E e); // throws InvalidPositionException;
    public Position<E> root(); // throws EmptyTreeException;
    public Position<E> parent(Position<E> v);
        // throws InvalidPositionException, BoundaryViolationException;
    public Iterable<Position<E>> children(Position<E> v); // throws InvalidPositionException;
    public boolean isInternal(Position<E> v); // throws InvalidPositionException;
    public boolean isExternal(Position<E> v); // throws InvalidPositionException;
    public boolean isRoot(Position<E> v); // throws InvalidPositionException;
    public void createRoot(E e); // throws InvalidOperationException;
    public Position<E> addFirstChild(Position<E> p, E e); // throws InvalidPositionException;
    public Position<E> addLastChild(Position<E> p, E e); // throws InvalidPositionException;
    public Position<E> addBefore(Position<E> p, Position<E> rb, E e);
        // throws InvalidPositionException;
    public Position<E> addAfter (Position<E> p, Position<E> lb, E e);
        // throws InvalidPositionException;
    public void removeExternalNode (Position<E> p); // throws InvalidPositionException;
    public void removeInternalNode (Position<E> p); // throws InvalidPositionException;
    public void removeNode (Position<E> p); // throws InvalidPositionException;
}

```

Estructuras de datos - Dr. Sergio A. Gómez

29

Ejemplo de carga de un árbol

```

Tree<Character> arbol = new Arbol<Character>(); // Creo un árbol de caracteres
arbol.createRoot( 'A' ); // Agrego la raíz
Position<Character> raiz = arbol.root();

```

// Agrego hijos de A:

```

Position<Character> pB = arbol.addLastChild( raiz, 'B' );
Position<Character> pC = arbol.addLastChild( raiz, 'C' );
Position<Character> pD = arbol.addLastChild( raiz, 'D' );

```

// Agrego hijos de B:

```

Position<Character> pH = arbol.addLastChild( pB, 'H' );
arbol.addLastChild( pB, 'F' ); // Ocurre un "voiding"

```

// Agrego los hijos de H:

```

arbol.addFirstChild( pH, 'G' );
arbol.addFirstChild( pH, 'K' );

```

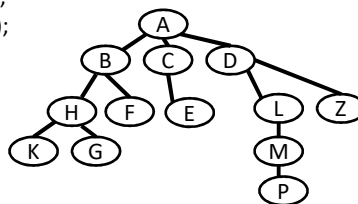
// Agrego hijo de C:

```

arbol.addLastChild( pC, 'E' ); // Completar con descendientes propios de D

```

NOTA: Veremos una implementación donde cada una de estas operaciones se realizan en tiempo constante en la cantidad de nodos del árbol.



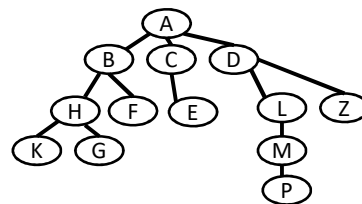
Estructuras de datos - Dr. Sergio A. Gómez

30

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Profundidad y altura

- Profundidad de un nodo v en un árbol T : Longitud del camino de la raíz de T al nodo v = cantidad de ancestros propios de v
- Longitud de un camino: Cantidad de arcos del camino
- Altura de un nodo v en un árbol T : Longitud del camino más largo a una hoja en el subárbol con raíz v .
- Altura de un árbol T : Altura del nodo raíz de T .
- Ej: profundidad de $A = 0$
- Ej: profundidad de $E = 2$
- Ej: profundidad de $P = 4$
- Ej: Altura de $B = 2$
- Ej: Altura de $D = 3$
- Ej: Altura de $G = 0$
- Ej: Profundidad de $G = 3$



Estructuras de datos - Dr. Sergio A. Gómez

31

Profundidad (depth)

Profundidad de un nodo v en un árbol T : Longitud del camino de la raíz de T al nodo v = cantidad de ancestros propios de v

$\text{Profundidad}(T, v) = \text{Longitud del camino de } v \text{ a } T.\text{root}()$

Algoritmo profundidad(T, v)

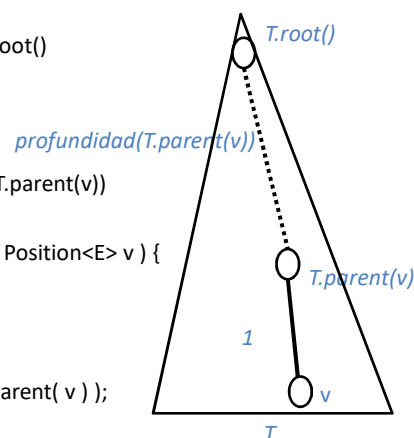
CB: Si v es la raíz de T , profundidad(T, v) = 0

CR: Si v no es la raíz de T ,

profundidad(T, v) = 1 + profundidad($T, T.\text{parent}(v)$)

Implementación Java:

```
public static <E> int profundidad( Tree<E> T, Position<E> v ) {
    if ( T.isRoot(v) )
        return 0;
    else
        return 1 + profundidad( T, T.parent( v ) );
}
```



Tiempo de ejecución: es del orden de d_v donde d_v es la profundidad de v en T

Estructuras de datos - Dr. Sergio A. Gómez

32

Profundidad iterativo

Algoritmo Profundidad(T, v):

```

    prof ← 0
    mientras v no es la raíz del árbol:
        incrementar prof
        v ← padre de v
    retornar prof

```

En Java:

```

public <E> int profundidad(Tree<E> t, Position<E> v) {
    int prof = 0;
    while (!t.isRoot(v)) {
        prof++;
        v = t.parent(v);
    }
    return prof;
}

```

Altura (Height) primera solución

Altura de un nodo v en un árbol T: Longitud del camino más largo a una hoja en el subárbol con raíz v.

Proposición: La altura de un árbol T no vacío es la máxima profundidad de las hojas de T

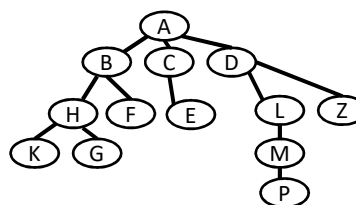
Algoritmo Altura(T)

```

h ← 0
para cada vértice v en T hacer
    si v es una hoja en T entonces
        h ← max( h, profundidad( T, v ) )
retornar h

```

En el árbol del ejemplo, la altura del árbol es el máximo entre las profundidades de K, G, F, E, P y Z.



Implementación Java:

```

public static <E> int altura( Tree<E> T ) {
    int h = 0;
    for( Position<E> v : T.positions() )
        if( T.isExternal(v) )
            h = Math.max( h, profundidad( T, v ) );
    return h;
}

```

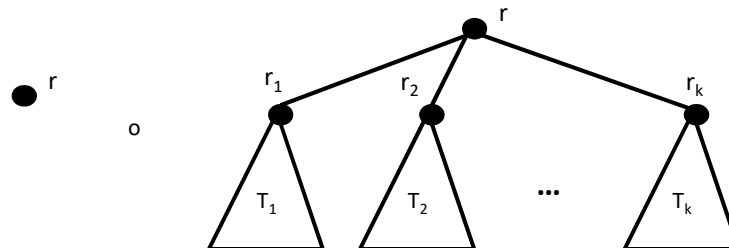
Tiempo de ejecución: Sea n = cantidad de nodos del árbol T

$T_{\text{altura}}(n) = O(1 + n \cdot \max(1, \text{altura_del_árbol})) = O(n \cdot n) = O(n^2)$

Definición recursiva de árbol no vacío

Un árbol no vacío T es:

- Hoja(r): Un nodo hoja r (llamado raíz de T)
- Nodo($r, T_1, T_2, \dots, T_k$): Es un nodo r (llamado la raíz de T) y un conjunto de árboles no vacíos $T_1, T_2, \dots, T_k$ cuyas raíces $r_1, r_2, \dots, r_k$ son los hijos de r .



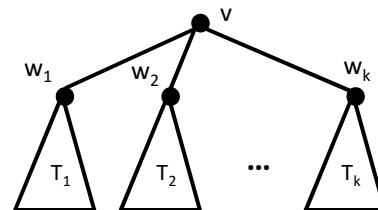
Estructuras de datos - Dr. Sergio A. Gómez

35

Altura (Height) segunda solución

- Altura de un nodo v en un árbol T : Longitud del camino más largo a una hoja en el subárbol con raíz v .
- Definición recursiva de la altura de v en T (planteo recursivo):
CB: altura(hoja(e)) = 0
CR: altura(nodo($e, t_1, \dots, t_k$)) = 1 + max(altura(t_1), ..., altura(t_k))

- Algoritmo Altura(T, v)
 si v es una hoja en T entonces
 Altura $\leftarrow 0$
 sino
 $h \leftarrow 0$
 para cada hijo w de v en T
 $h \leftarrow \max(h, \text{Altura}(T, w))$
 Altura $\leftarrow 1+h$



$$\text{Altura}(T, v) = 1 + \max(\text{altura}(T, v_1), \dots, \text{altura}(T, v_k))$$

Tiempo de ejecución: Si $n = T.\text{size}()$, $T_{\text{altura}}(n) = O(n)$

Estructuras de datos - Dr. Sergio A. Gómez

36

Altura (Height) segunda solución

Algoritmo Altura(T,v)

```

si v es una hoja en T entonces
    Altura ← 0
sino
    h ← 0
    para cada hijo w de v en T
        h ← max( h, Altura( T, w ) )
    Altura ← 1+h

```

Implementación Java: Llamar con altura(T, T.root()) para altura de T

```

public static <E> int altura( Tree<E> T, Position<E> v ) {
    if( T.isExternal(v) )
        return 0;
    else {
        int h = 0;
        for( Position<E> w : T.children(v) )
            h = Math.max( h, altura( T, w ) );
        return 1+h;
    }
}

```

Estructuras de datos - Dr. Sergio A. Gómez

37

Tiempo de segunda solución de altura

Algoritmo Altura(T,v)

```

si v es una hoja en T entonces
    Altura ← 0
sino
    h ← 0
    para cada hijo w de v en T
        h ← max( h, Altura( T, w ) )
    Altura ← 1+h

```

- Entrada: un árbol T
- Tamaño de la entrada: n = cantidad de nodos del árbol T
- Tiempo de ejecución: Vemos que el algoritmo pasa una vez por cada nodo i y en el nodo toma un tiempo constante c_2 y luego ejecuta un bucle que realiza h_i iteraciones, con h_i = la cantidad de hijos del nodo i.

$$T(n) = c_1 + \sum_{i=1}^n (c_2 + c_3 h_i) = c_1 + \sum_{i=1}^n (c_2) + \sum_{i=1}^n (c_3 h_i) = c_1 + \sum_{i=1}^n (c_2) + c_3 \sum_{i=1}^n (h_i) = c_1 + c_2 n + c_3 (n-1) = O(n)$$

Notar que $\sum_{i=1}^n h_i = n - 1$ pues corresponde a la cantidad de arcos del árbol.

La expresión $\sum_{i=1}^n h_i$ corresponde a la cantidad de hijos del nodo1 más la cantidad de hijos del nodo2 más la cantidad de hijos del nodo3 y así sucesivamente, lo que da la cantidad de arcos del árbol. Como todos los nodos tienen un único padre menos la raíz que no tiene, entonces si hay n nodos, entonces la cantidad de arcos del árbol es n-1.

Estructuras de datos - Dr. Sergio A. Gómez

38

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Recorridos de árboles

- Un recorrido de un árbol T es una manera sistemática de visitar todos los nodos de T.
- Los recorridos básicos son:
 - Preorden (orden previo)
 - Postorden (orden posterior)
 - Inorden (orden simétrico)
 - Por niveles (level ordering)

Recorridos con Tour de Euler

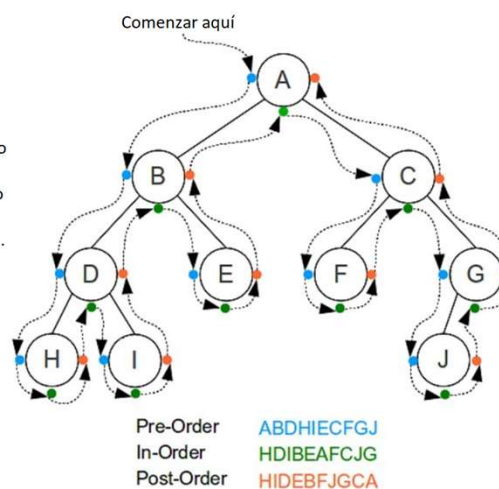
Tour de Euler para encontrar listados preorden, postorden e inorden:

* Preorden: Listar el nodo la primera vez que lo visito

* Postorden: Listar el nodo la última vez que lo visito

* Inorden: Listar el nodo la segunda vez que lo visito.

Ver página 348 (sección 8.4.6 de 6ta edición de GT)



Recorridos de árboles: Preorden

- En el recorrido preorden de un árbol T , la raíz r de T se visita primero y luego se visitan recursivamente cada uno de los subárboles de r .
- Si el árbol es ordenado los subárboles se visitan en el orden en el que aparecen.
- El algoritmo se llama con `preorden( T, T.root() )`
- Algoritmo `preorden( T, v )`

`Visitar(T, v)`

Para cada hijo w de v en T :

`preorden( T, w )`

- Planteo Preorden:
 - CB: `pre(hoja(n)) = [n]`
 - CR: `pre( nodo(n, [t1 t2 ... tk]) ) =`
 $[n] + \text{pre}(t_1) + \text{pre}(t_2) + \dots + \text{pre}(t_k)$
- La acción “`visitar(T,v)`” dependerá del problema.

Entendiendo preorden

Primero veamos un problema más fácil, imprimir una lista no vacía recursivamente:

```
imprimir(L):
    imprimir_rec(L, L.first())

imprimir_rec(L, p):
    if p = L.last():
        print(p.element())
    else:
        print(p.element())
        imprimir_rec(L, L.next(p))
```

Cómo la primera instrucción del then y else es la misma se puede factorizar:

```
imprimir_rec(L, p):
    print(p.element())
    if p ≠ L.last():
        imprimir_rec(L, L.next(p))
```

Con listas este problema es fácil de resolver porque cada nodo excepto el último tiene exactamente un único siguiente.

En el caso del árbol hay un siguiente por cada hijo de P salvo que P sea el último de un camino que se inicia en la raíz y llega a una hoja P .

Origen de código de preorden

Pre(t, v):

Se plantea recursivamente por casos (asumo que visita(v) es imprimir)

CB: Si v es una hoja => imprimir rótulo de v

CR: Si v no es una hoja => imprimir rótulo de v y listar preorden cada uno de los hijos de v

Planteo expresado en pseudocódigo: Pre(t,v):

si v es hoja: imprimir rótulo de v

sino:

imprimir rótulo v

para cada hijo h de v: pre(t, h)

Refactorizo: (saco *imprimir* fuera del condicional)

imprimir rótulo de v

si v es hoja:

sino: para cada hijo h de v: pre(t, h)

Refactorizo: (Elimino el then vacío)

imprimir rótulo de v

si no (v es hoja): para cada hijo h de v: pre(t, h)

Refactorizo: (Como cuando v es hoja no tiene hijos => el para de los hijos no hace iteraciones)

imprimir rótulo de v

para cada hijo h de v: pre(t, h)

Estructuras de datos - Dr. Sergio A. Gómez

43

Recorridos en árboles: preorden

Algoritmo PreordenShell(T)

preorden(T, T.root())

Algoritmo preorden(T, v)

Visitar(T, v)

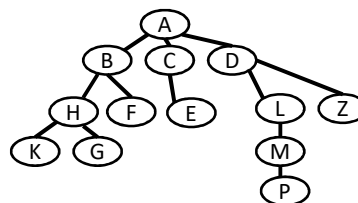
Para cada hijo w de v en T hacer

preorden(T, w)

• Planteo Preorden:

CB: pre(hoja(n)) = [n]

CR: pre(nodo(n, [t₁ t₂ ... t_k])) =
[n] + pre(t₁) + pre(t₂) + ... + pre(t_k)



Ejemplo: El recorrido preorden es:

A - B - H - K - G - F - C - E - D - L - M - P - Z

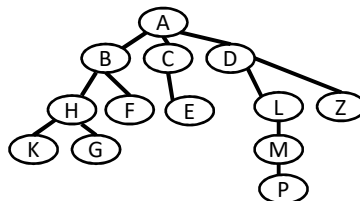
Estructuras de datos - Dr. Sergio A. Gómez

44

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida:

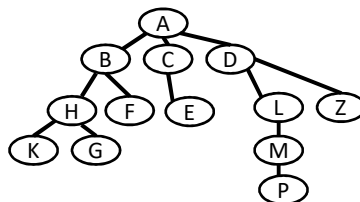
Preshell(T) => pre(A)

PreordenShell llama a preorden con A.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A

Preshell(T) => pre(A)

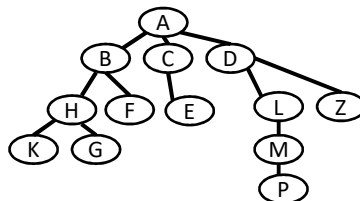
pre(A) => pre(B)

Preorden de A imprime A y llama a preorden con B.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B

Preshell(T) => pre(A)

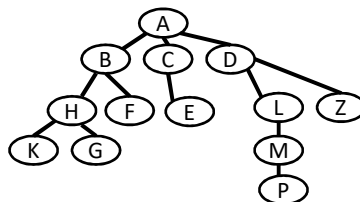
pre(A) => pre(B) => pre(H)

Preorden de B imprime B y llama a preorden con H.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H

Preshell(T) => pre(A)

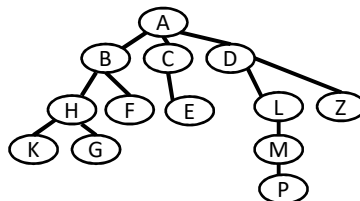
pre(A) => pre(B) => pre(H) => pre(K)

Preorden de H imprime H y llama a preorden de K.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K

Preshell(T) => pre(A)

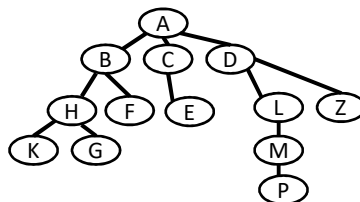
pre(A) => pre(B) => pre(H) => pre(K)

Preorden de K imprime K y como K es una hoja, termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K

Preshell(T) => pre(A)

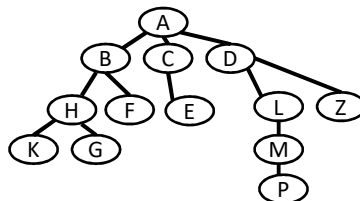
pre(A) => pre(B) => pre(H)

Vuelve el control a preorden de H, busca el siguiente hijo G, llamando a preorden de G.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K

Preshell(T) => pre(A)

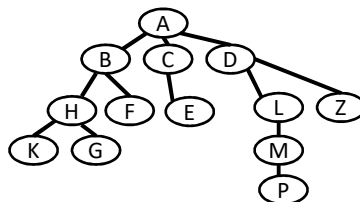
pre(A) => pre(B) => pre(H) => pre(G)

Se crea el registro de activación de preorden de G.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G

Preshell(T) => pre(A)

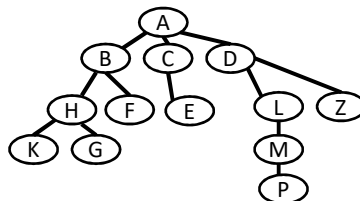
pre(A) => pre(B) => pre(H) => pre(G)

Preorden de G imprime G y como es hoja, termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G

Preshell(T) => pre(A)

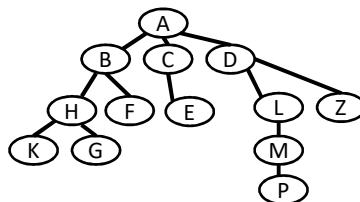
pre(A) => pre(B) => pre(H)

El control volvió a preorden de H, pero como no tiene más hijos, termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G

Preshell(T) => pre(A)

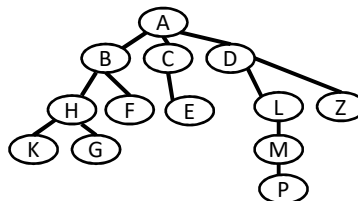
pre(A) => pre(B)

El control volvió a preorden de B, busca el siguiente hijo F y llama a preorden de F.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G

Preshell(T) => pre(A)

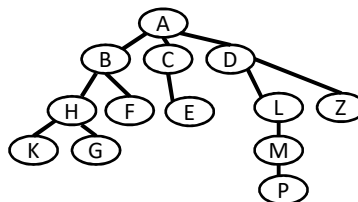
pre(A) => pre(B) => pre(F)

Se pasó el control a preorden de F creándose un registro de activación.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F

Preshell(T) => pre(A)

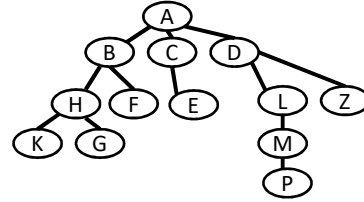
pre(A) => pre(B) => pre(F)

Preorden de F imprime F y como no tiene hijos, termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F

Preshell(T) => pre(A)

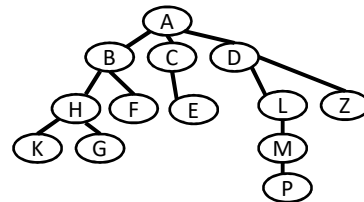
pre(A) => pre(B)

Vuelve el control a preorden de B pero como no tiene más hijos, termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F

Preshell(T) => pre(A)

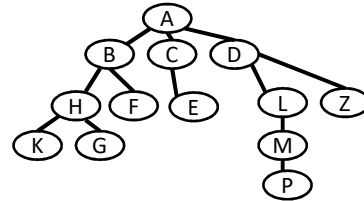
pre(A)

Vuelve el control a preorden de A, busca el siguiente hijo C y llama a preorden de C.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F

Preshell(T) => pre(A)

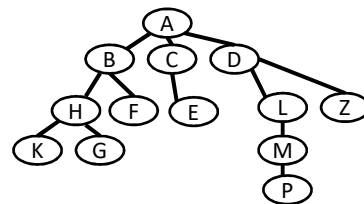
pre(A) => pre(C)

Se le pasa el control a preorden de C creando un nuevo registro de activación.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C

Preshell(T) => pre(A)

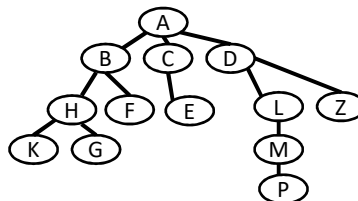
pre(A) => pre(C)

Preorden de C imprime C y busca su primer hijo E, llamando a preorden de E.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C

Preshell(T) => pre(A)

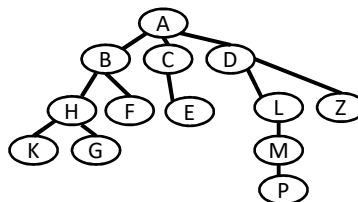
pre(A) => pre(C) => pre(E)

Se le pasa el control a preorden de E creando un nuevo registro de activación.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E

Preshell(T) => pre(A)

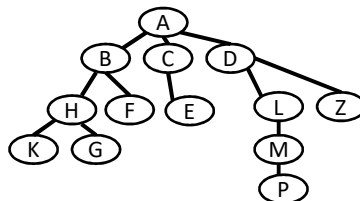
pre(A) => pre(C) => pre(E)

Preorden de E imprime E y como no tiene hijos termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E

Preshell(T) => pre(A)

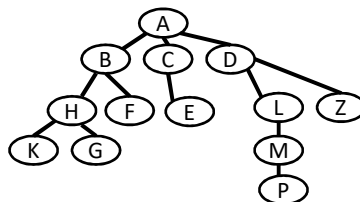
pre(A) => pre(C)

Vuelve el control a preorden de C pero como no tiene más hijos, termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E

Preshell(T) => pre(A)

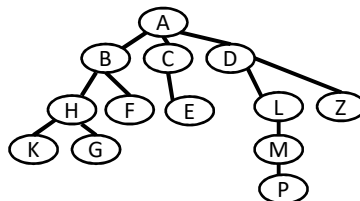
pre(A)

Vuelve el control a preorden de A y busca el siguiente hijo D, llamando a preorden de D.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E

Preshell(T) => pre(A)

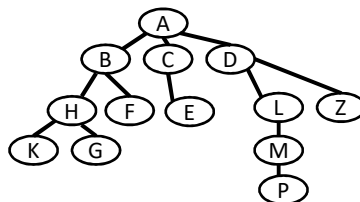
pre(A) => pre(D)

Se crea un nuevo registro de activación y ahora el control está en preorden de D.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E D

Preshell(T) => pre(A)

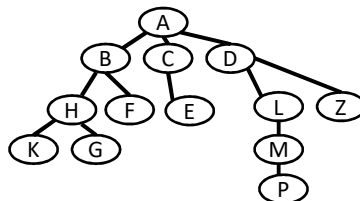
pre(A) => pre(D)

Preorden de D imprime D y busca su primer hijo L, llamando a preorden de L.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z
Salida: A B H K G F C E D

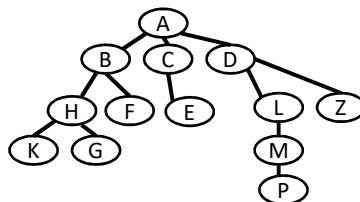
Preshell(T) => pre(A)
pre(A) => pre(D) => pre(L)

Se crea un registro de activación para preorden de L y ahora tiene el control.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z
Salida: A B H K G F C E D L

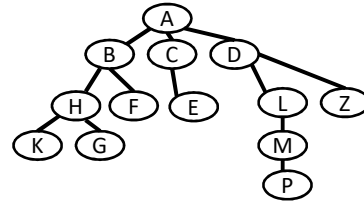
Preshell(T) => pre(A)
pre(A) => pre(D) => pre(L)

Preorden de L imprime L y busca su primer hijo M, llamando a preorden de M.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E D L

Preshell(T) => pre(A)

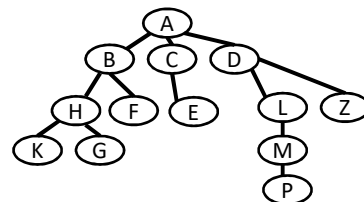
pre(A) => pre(D) => pre(L) => pre(M)

Ahora se creó un registro de activación para preorden de M quien tiene el control.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E D L M

Preshell(T) => pre(A)

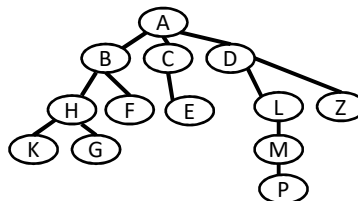
pre(A) => pre(D) => pre(L) => pre(M)

Preorden de M imprime M y busca su primer hijo P, llamando a preorden de P.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E D L M

Preshell(T) => pre(A)

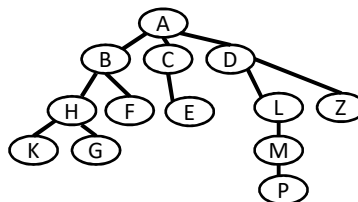
pre(A) => pre(D) => pre(L) => pre(M) => pre(P)

Se crea un registro de activación para preorden de P, quien ahora tiene el control.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E D L M P

Preshell(T) => pre(A)

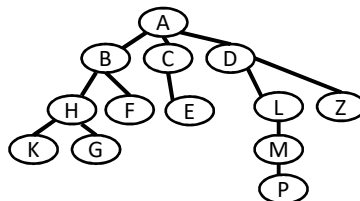
pre(A) => pre(D) => pre(L) => pre(M) => pre(P)

Preorden de P imprime P y como es una hoja, termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E D L M P

Preshell(T) => pre(A)

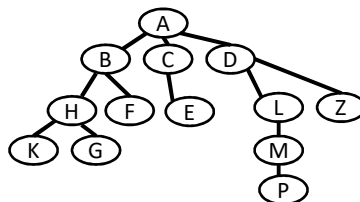
pre(A) => pre(D) => pre(L) => pre(M)

El control está en preorden de M nuevamente, pero como no tiene más hijos, termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E D L M P

Preshell(T) => pre(A)

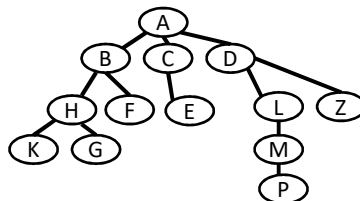
pre(A) => pre(D) => pre(L)

Ahora el control está de nuevo en preorden de L, quien como no tiene más hijos, termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E D L M P

Preshell(T) => pre(A)

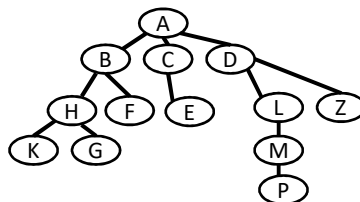
pre(A) => pre(D)

El control vuelve a preorden de D, quien busca el siguiente hijo Z y llama a preorden de Z.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E D L M P

Preshell(T) => pre(A)

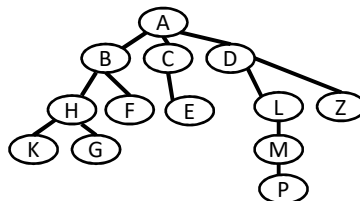
pre(A) => pre(D) => pre(Z)

Se crea un registro de activación para preorden de Z.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z
Salida: A B H K G F C E D L M P Z

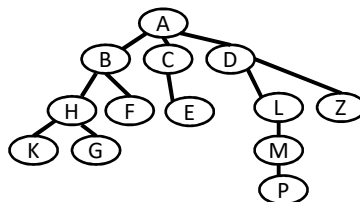
Preshell(T) => pre(A)
pre(A) => pre(D) => pre(Z)

Preorden de Z imprime Z y como no tiene hijos, termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z
Salida: A B H K G F C E D L M P Z

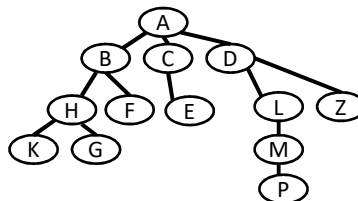
Preshell(T) => pre(A)
pre(A) => pre(D)

El control vuelve a preorden de D, quien no tiene más hijos, entonces termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z
Salida: A B H K G F C E D L M P Z

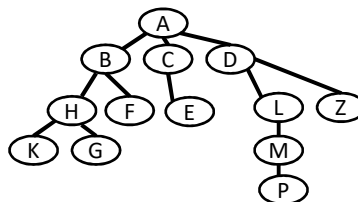
Preshell(T) => pre(A)
pre(A)

El control vuelve a preorden de A, que no tiene más hijos, termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z
Salida: A B H K G F C E D L M P Z

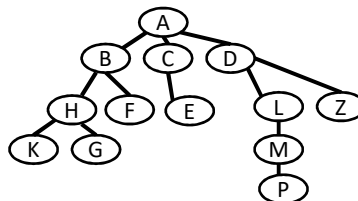
Preshell(T)

El control vuelve a preordenshell, quien termina y sale.

Recorridos en árboles: preorden -- traza

Algoritmo PreordenShell(T)
preorden(T, T.root())

Algoritmo preorden(T, v)
Visitar(T, v)
Para cada hijo w de v en T hacer
preorden(T, w)



Resultado esperado: A B H K G F C E D L M P Z

Salida: A B H K G F C E D L M P Z

La traza fue exitosa porque la salida del programa coincide con el resultado esperado.

Tiempo de ejecución de preorden

- Entrada: un árbol T
- Tamaño de la entrada: n = cantidad de nodos del árbol T
- Tiempo de ejecución: Vemos que el algoritmo pasa una vez por cada nodo i y en el nodo toma un tiempo constante c_2 y luego ejecuta un bucle que realiza h_i iteraciones, con h_i = la cantidad de hijos del nodo i .

$$T(n) = c_1 + \sum_{i=1}^n (c_2 + c_3 h_i) =$$

$$= c_1 + c_2 n + c_3 (n-1) = O(n)$$

Notar que $\sum_{i=1}^n h_i = n - 1$ pues corresponde a la cantidad de arcos del árbol.

La expresión $\sum_{i=1}^n h_i$ corresponde a la cantidad de hijos del nodo1 más la cantidad de hijos del nodo2 más la cantidad de hijos del nodo3 y así sucesivamente, lo que da la cantidad de arcos del árbol. Como todos los nodos tienen un único padre menos la raíz que no tiene, entonces si hay n nodos, entonces la cantidad de arcos del árbol es $n-1$.

Recorridos en árboles: preorden

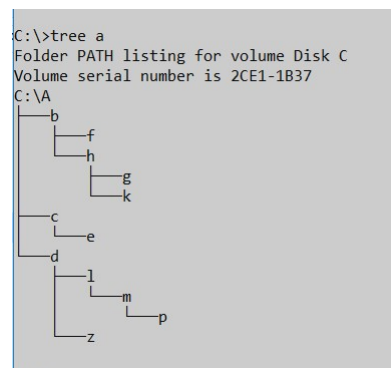
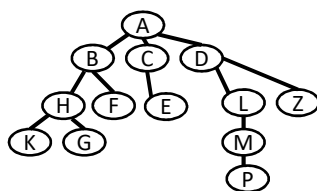
- Aplicación: El recorrido preorden de un libro a partir de su tabla de contenidos examina el libro en forma secuencial.
- Aplicación: Retornar el listado preorden como un string.

```
public String toStringPreorder ( Tree<E> T, Position<E> v ) {
    String s = v.element().toString();
    for( Position<E> w : T.children(v) )
        s += " - " + toStringPreorder( T, w );
    return s;
}
```

Nota: Si n es la cantidad de nodos del árbol T , el tiempo de ejecución de preorden es $O(n)$ asumiendo una visita de $O(1)$.

Es decir, preorden corre en tiempo lineal en el tamaño del árbol.

Ejemplo de preorden



El sistema operativo ordena las carpetas en orden alfabético.

El comando *tree* realiza un listado preorden de las carpetas aumentando la profundidad hacia la derecha.

Recorridos de árboles: Postorden

- En el recorrido postorden de un árbol T , la raíz r de T se visita luego de visitar recursivamente cada uno de los subárboles de r .
- Si el árbol es ordenado los subárboles se visitan en el orden en el que aparecen.
- El algoritmo se llama con $\text{postorden}(T, T.\text{root}())$
- Algoritmo $\text{postorden}(T, v)$
 - Para cada hijo w de v en T hacer
 - $\text{postorden}(T, w)$
 - $\text{Visitar}(T, v)$
- Planteo Postorden:
 - CB: $\text{post}(\text{hoja}(n)) = [n]$
 - CR: $\text{post}(\text{nodo}(n, [t_1 \ t_2 \ \dots \ t_k])) = \text{post}(t_1) + \text{post}(t_2) + \dots + \text{post}(t_k) + [n]$
- La acción “ $\text{visitar}(T, v)$ ” dependerá del problema.

Recorridos en árboles: postorden

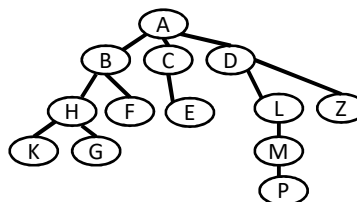
Algoritmo $\text{PostordenShell}(T)$
 $\text{postorden}(T, T.\text{root}())$

Algoritmo $\text{postorden}(T, v)$
 Para cada hijo w de v en T hacer
 $\text{postorden}(T, w)$
 $\text{Visitar}(T, v)$

Ejemplo: El recorrido postorden es:
 K, G, H, F, B, E, C, P, M, L, Z, D, A

Tiempo de ejecución: Si el árbol T tiene n nodos entonces:
 $T_{\text{postorden}}(n) = O(n)$ asumiendo visita de $O(1)$

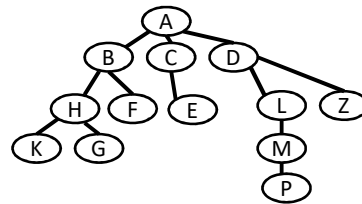
• Planteo Postorden:
CB: $\text{post}(\text{hoja}(n)) = [n]$
CR: $\text{post}(\text{nodo}(n, [t_1 \ t_2 \ \dots \ t_k])) = \text{post}(t_1) + \text{post}(t_2) + \dots + \text{post}(t_k) + [n]$



Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida:

Postshell(T) => post(A)

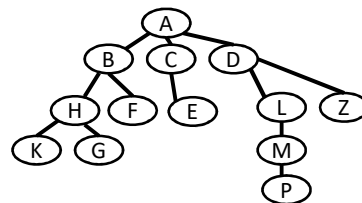
post(A)

PostordenShell llama a postorden con A.

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida:

Postshell(T) => post(A)

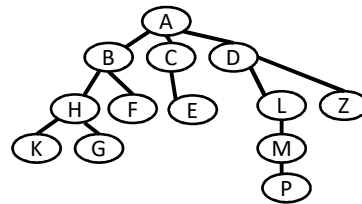
post(A) => post(B)

Postorden con A busca el primer hijo y llama a postorden de B.

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida:

Postshell(T) => post(A)

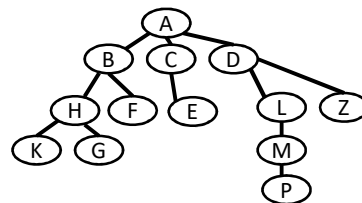
post(A) => post(B) => post(H)

Postorden con B busca el primer hijo y llama a postorden de H.

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida:

Postshell(T) => post(A)

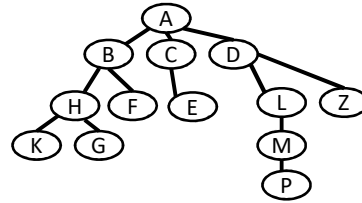
post(A) => post(B) => post(H) => post(K)

Postorden con H busca el primer hijo y llama a postorden de K.

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K

Postshell(T) => post(A)

post(A) => post(B) => post(H) => post(K)

Postorden con K no encuentra hijos, imprime K y sale.

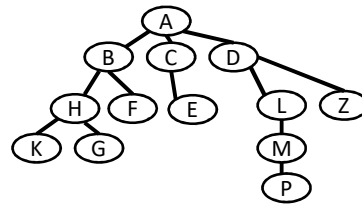
Estructuras de datos - Dr. Sergio A. Gómez

91

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K

Postshell(T) => post(A)

post(A) => post(B) => post(H)

El control está de nuevo en postorden de H, quien busca al siguiente hijo G y llama a postorden de G.

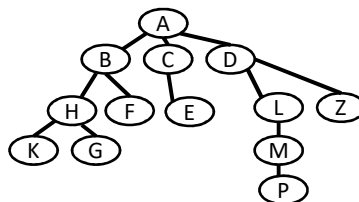
Estructuras de datos - Dr. Sergio A. Gómez

92

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G

Postshell(T) => post(A)

post(A) => post(B) => post(H) => post(G)

Se crea un registro de activación para postorden de G, quien no encuentra hijos, imprime G y sale.

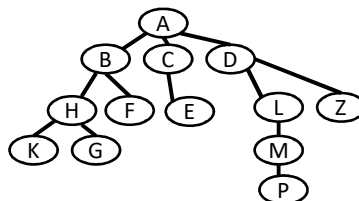
Estructuras de datos - Dr. Sergio A. Gómez

93

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H

Postshell(T) => post(A)

post(A) => post(B) => post(H)

*El control está en postorden de H nuevamente.
 Como no encuentra más hijos, imprime H y sale.*

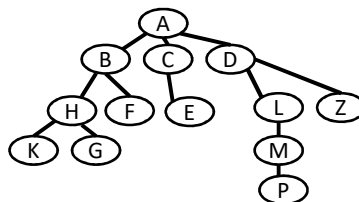
Estructuras de datos - Dr. Sergio A. Gómez

94

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H

Postshell(T) => post(A)

post(A) => post(B)

El control está en postorden de B nuevamente.

Busca el siguiente hijo F y llama postorden de F.

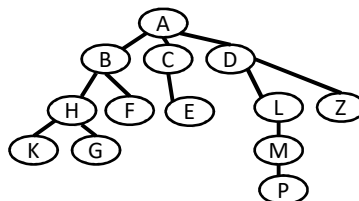
Estructuras de datos - Dr. Sergio A. Gómez

95

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H

Postshell(T) => post(A)

post(A) => post(B) => post(F)

Se crea un registro de activación para postorden de F quien tiene el control ahora.

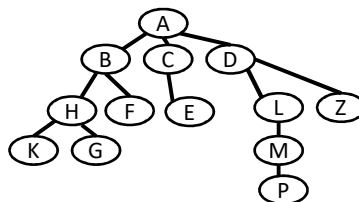
Estructuras de datos - Dr. Sergio A. Gómez

96

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F

Postshell(T) => post(A)

post(A) => post(B) => post(F)

Postorden de F no encuentra hijos, imprime F y sale.

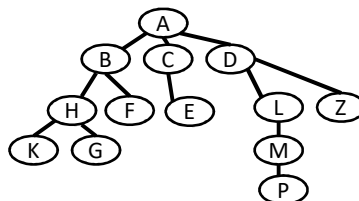
Estructuras de datos - Dr. Sergio A. Gómez

97

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B

Postshell(T) => post(A)

post(A) => post(B)

El control vuelve a postorden de B, que como no tiene más hijos que explorar, imprime B y sale.

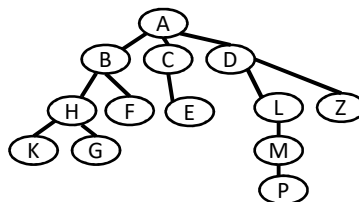
Estructuras de datos - Dr. Sergio A. Gómez

98

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B

Postshell(T) => post(A)

post(A)

El control vuelve a postorden de A, que sigue recorriendo hijos y encuentra a C, entonces llama a postorden de C.

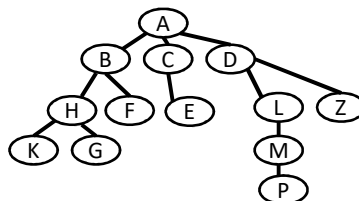
Estructuras de datos - Dr. Sergio A. Gómez

99

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B

Postshell(T) => post(A)

post(A) => post(C)

El control vuelve a postorden de A, que sigue recorriendo hijos y encuentra a C, entonces llama a postorden de C.

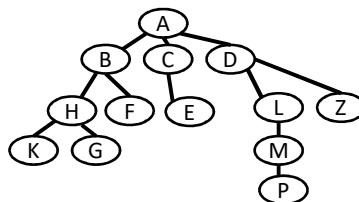
Estructuras de datos - Dr. Sergio A. Gómez

100

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B

Postshell(T) => post(A)

post(A) => post(C)

*El control está ahora en postorden de C, que recorre sus hijos encontrando a E.
 Llama entonces a postorden de E.*

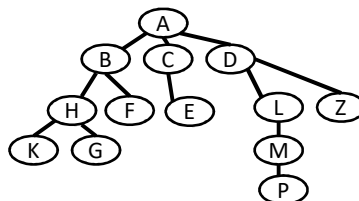
Estructuras de datos - Dr. Sergio A. Gómez

101

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E

Postshell(T) => post(A)

post(A) => post(C) => post(E)

El control está ahora en postorden de E, que recorre sus hijos y no encuentra a ninguno porque E es una hoja. Imprime E y sale.

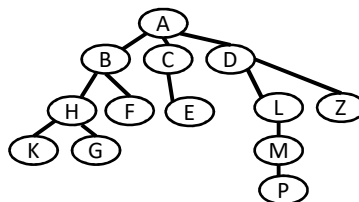
Estructuras de datos - Dr. Sergio A. Gómez

102

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C

Postshell(T) => post(A)

post(A) => post(C)

El control está de nuevo en C, que como no tiene más hijos para ver, imprime C y sale.

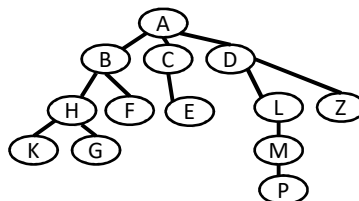
Estructuras de datos - Dr. Sergio A. Gómez

103

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C

Postshell(T) => post(A)

post(A)

El control está de nuevo en A, que recupera al último hijo D y llama a postorden con D.

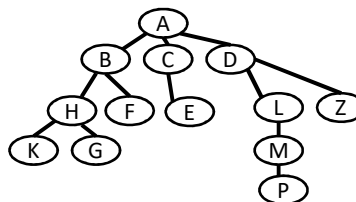
Estructuras de datos - Dr. Sergio A. Gómez

104

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C

Postshell(T) => post(A)

post(A) => post(D)

El control está de nuevo en A, que recupera al último hijo D y llama a postorden con D.

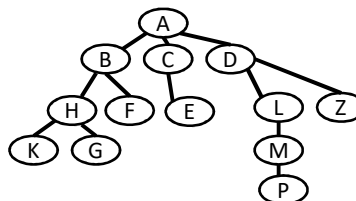
Estructuras de datos - Dr. Sergio A. Gómez

105

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C

Postshell(T) => post(A)

post(A) => post(D)

El control está en D, quien encuentra un hijo L y llama a postorden de L.

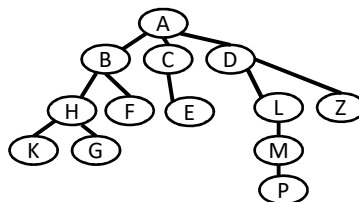
Estructuras de datos - Dr. Sergio A. Gómez

106

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C

Postshell(T) => post(A)

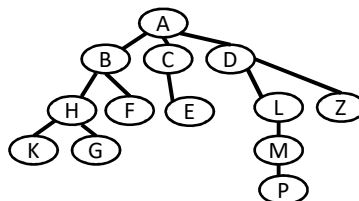
post(A) => post(D) => post(L)

El control está en L, quien encuentra un hijo M y llama a postorden de M.

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C

Postshell(T) => post(A)

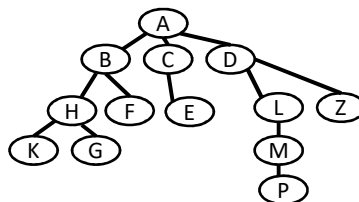
post(A) => post(D) => post(L) => post(M)

El control está en M, quien encuentra un hijo P y llama a postorden de P.

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C P

Postshell(T) => post(A)

post(A) => post(D) => post(L) => post(M) => post(P)

*El control está en P, quien no encuentra hijos porque P es una hoja.
 Imprime P y sale.*

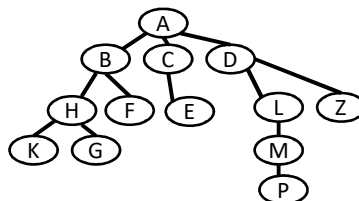
Estructuras de datos - Dr. Sergio A. Gómez

109

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C P M

Postshell(T) => post(A)

post(A) => post(D) => post(L) => post(M)

*El control está de nuevo en M, quien no encuentra más hijos.
 Imprime M y sale.*

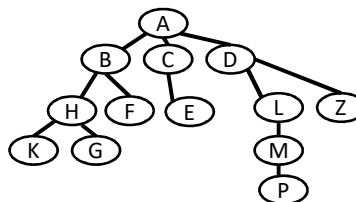
Estructuras de datos - Dr. Sergio A. Gómez

110

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C P M L

Postshell(T) => post(A)

post(A) => post(D) => post(L)

El control está de nuevo en L, quien no encuentra más hijos.

Imprime L y sale.

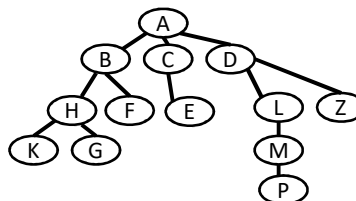
Estructuras de datos - Dr. Sergio A. Gómez

111

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C P M L

Postshell(T) => post(A)

post(A) => post(D) => post(L)

El control está de nuevo en L, quien no encuentra más hijos.

Imprime L y sale.

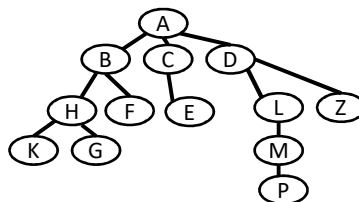
Estructuras de datos - Dr. Sergio A. Gómez

112

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C P M L

Postshell(T) => post(A)

post(A) => post(D)

*El control está de nuevo en D, quien ahora encuentra otro hijo llamado Z.
 Llama a postorden de Z.*

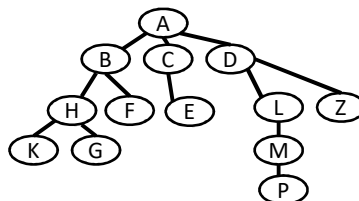
Estructuras de datos - Dr. Sergio A. Gómez

113

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C P M L Z

Postshell(T) => post(A)

post(A) => post(D) => post(Z)

*El control está en Z, quien no tiene hijos.
 Imprime Z y sale.*

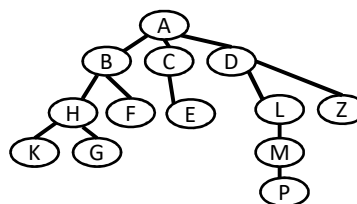
Estructuras de datos - Dr. Sergio A. Gómez

114

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C P M L Z D

Postshell(T) => post(A)

post(A) => post(D)

*El control está en D nuevamente, quien no tiene más hijos para explorar
 Imprime D y sale.*

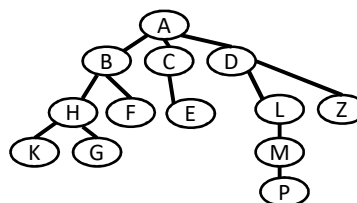
Estructuras de datos - Dr. Sergio A. Gómez

115

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C P M L Z D A

Postshell(T) => post(A)

post(A)

*El control está en A nuevamente, quien no tiene más hijos para explorar.
 Imprime A y sale.*

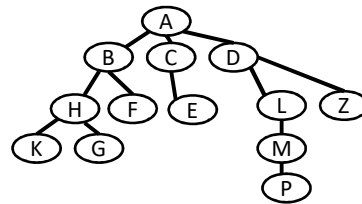
Estructuras de datos - Dr. Sergio A. Gómez

116

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C P M L Z D A

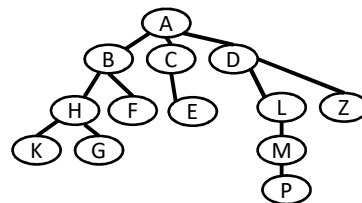
Postshell(T)

El control está en Postordenshell nuevamente, quien sale.

Recorridos en árboles: postorden -- traza

Algoritmo PostordenShell(T)
 postorden(T, T.root())

Algoritmo postorden(T, v)
 Para cada hijo w de v en T hacer
 postorden(T, w)
 Visitar(T, v)



Resultado esperado: K, G, H, F, B, E, C, P, M, L, Z, D, A

Salida: K G H F B E C P M L Z D A

Vemos que la salida del programa coincide con la salida esperada.

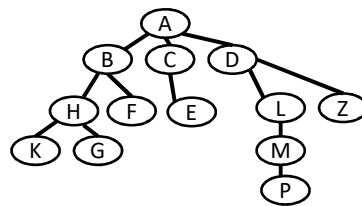
Recorridos de árboles: Inorden

- En el recorrido inorden (o simétrico) de un árbol T con raíz r , primero se recorre recursivamente el primer hijo de la raíz r , luego se visita la raíz y luego se visita recursivamente al resto de los hijos de r .
- Si el árbol es ordenado los subárboles se visitan en el orden en el que aparecen.
- El algoritmo se llama con $\text{inorden}(T, T.\text{root}())$
- Algoritmo $\text{inorden}(T, v)$
 - Si v es hoja en T entonces
 - Visitar(T, v)
 - Sino
 - $w \leftarrow$ primer hijo de v en T
 - $\text{inorden}(T, w)$
 - Visitar(T, v)
 - mientras w tiene hermano en T hacer
 - $w \leftarrow$ hermano de w en T
 - $\text{inorden}(T, w)$
- El recorrido inorden para el ejemplo es:
K H G B F A E C P M L D Z

• Planteo In order:

CB: $\text{in}(\text{hoja}(n)) = [n]$

CR: $\text{in}(\text{nodo}(n, [t_1 t_2 \dots t_k])) =$
 $\text{in}(t_1) + [n] + \text{in}(t_2) + \dots + \text{in}(t_k)$

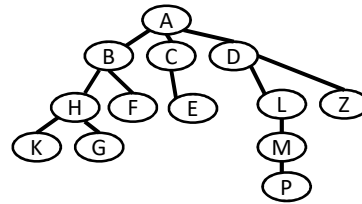


Estructuras de datos - Dr. Sergio A. Gómez

119

Recorridos de árboles: Inorden -- Traza

- El algoritmo se llama con $\text{inorden}(T, T.\text{root}())$
- Algoritmo $\text{inorden}(T, v)$
 - Si v es hoja en T entonces
 - Visitar(T, v)
 - Sino
 - $w \leftarrow$ primer hijo de v en T
 - $\text{inorden}(T, w)$
 - Visitar(T, v)
 - mientras w tiene hermano en T hacer
 - $w \leftarrow$ hermano de w en T
 - $\text{inorden}(T, w)$



Salida esperada: K H G B F A E C P M L D Z

Salida:

Inorder(T)

Se invoca a inorder, que llama a inorder con A.

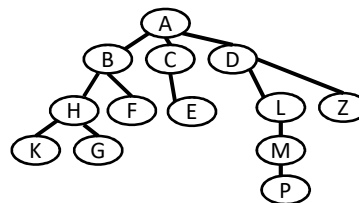
Estructuras de datos - Dr. Sergio A. Gómez

120

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - `Visitar( T, v )`
 - Sino
 - `w ← primer hijo de v en T`
 - `inorden( T, w )`
 - `Visitar( T, v )`
 - mientras `w` tiene hermano en `T` hacer
 - `w ← hermano de w en T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida:

`Inorder(T) => in(A)`

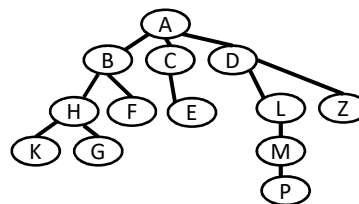
Se crea el registro de activación y el control pasa a A.

Como A no es hoja, y el primer hijo de A es B, se llama a inorden de B.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - `Visitar( T, v )`
 - Sino
 - `w ← primer hijo de v en T`
 - `inorden( T, w )`
 - `Visitar( T, v )`
 - mientras `w` tiene hermano en `T` hacer
 - `w ← hermano de w en T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida:

`Inorder(T) => in(A) => in(B)`

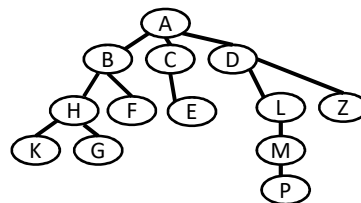
Se crea el registro de activación y el control pasa a B.

Como B no es hoja y el primer hijo de B es H, se llama a inorden de H.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida:

`Inorder(T) => in(A) => in(B) => in(H)`

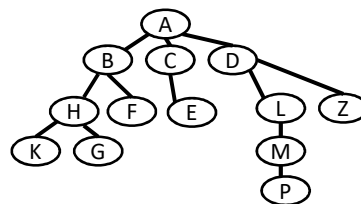
Se crea el registro de activación y el control pasa a H.

Como H no es hoja y el primer hijo de H es K, se llama a inorden de K.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K

`Inorder(T) => in(A) => in(B) => in(H) => in(K)`

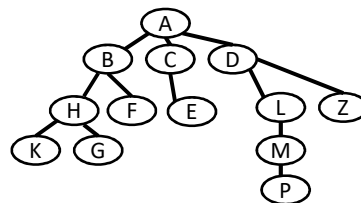
Se crea el registro de activación y el control pasa a K.

Como K sí es hoja, se imprime K y sale.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H

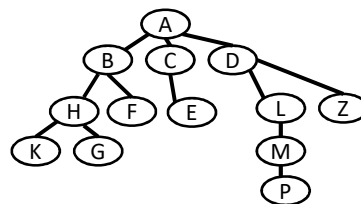
`Inorder(T) => in(A) => in(B) => in(H)`

*El control vuelve a H, imprime H y explora su 2do hijo encontrando a G.
Llama a inorden de G.*

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G

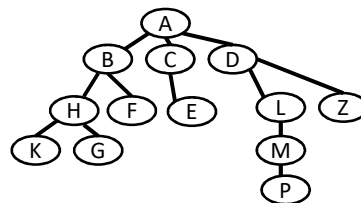
`Inorder(T) => in(A) => in(B) => in(H) => in(G)`

*Se crea el registro de activación para inorden de G.
Como G es una hoja, imprime G y sale.*

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G

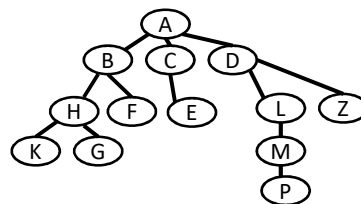
`Inorder(T) => in(A) => in(B) => in(H)`

El control está en H nuevamente, pero como no tiene más hijos que explorar, sale.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B

`Inorder(T) => in(A) => in(B)`

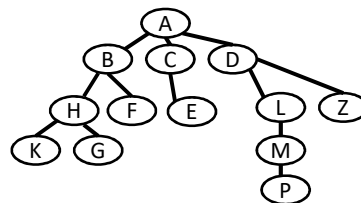
El control está en B nuevamente.

Imprime B y llama a inorden con el siguiente hijo, que es F.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B

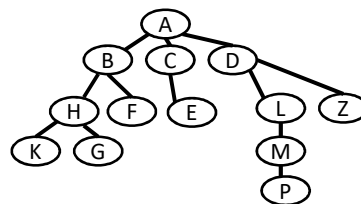
`Inorder(T) => in(A) => in(B) => in(F)`

*Se crea un registro de activación para inorden de F.
Como F es hoja, imprime el valor F y sale.*

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F

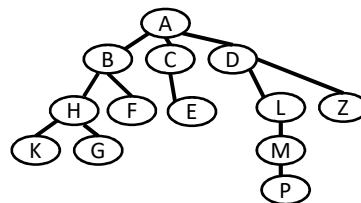
`Inorder(T) => in(A) => in(B)`

*El control está en B nuevamente.
Como B no tiene más hijos para explorar, sale.*

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A

`Inorder(T) => in(A)`

El control está ahora de nuevo en A.

Se imprime A y se busca el 2do hijo de A, que es C.

Se llama a inorden de C.

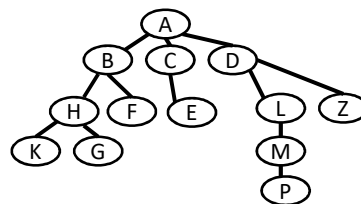
Estructuras de datos - Dr. Sergio A. Gómez

131

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A

`Inorder(T) => in(A) => in(C)`

Se crea un registro de activación para inorden de C.

Como C tiene un hijo, que es E, se llama a inorden de E.

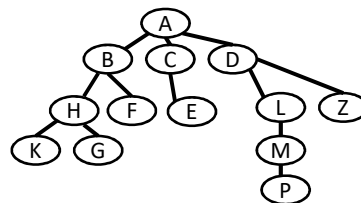
Estructuras de datos - Dr. Sergio A. Gómez

132

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A

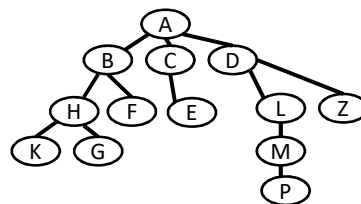
`Inorder(T) => in(A) => in(C) => in(E)`

*Se crea un registro de activación para inorden de E.
Como E es una hoja, se imprime E sale.*

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C

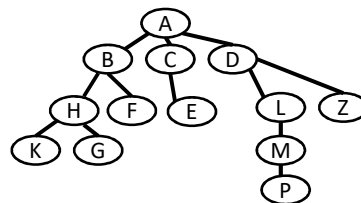
`Inorder(T) => in(A) => in(C)`

*El control está en C de nuevo, se imprime C.
Como no tiene más hijos, sale.*

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C

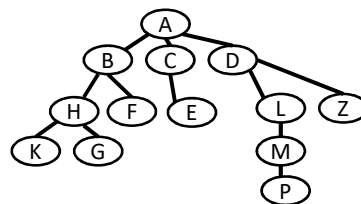
`Inorder(T) => in(A)`

*El control está en A de nuevo, se busca el siguiente hijo, que resulta ser D.
Se llama a inorden de D.*

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C

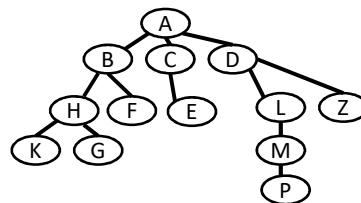
`Inorder(T) => in(A) => in(D)`

*Se crea un registro de activación para inorden de D.
Como D tiene un hijo, que se llama L, se llama a inorden de L.*

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C

`Inorder(T) => in(A) => in(D) => in(L)`

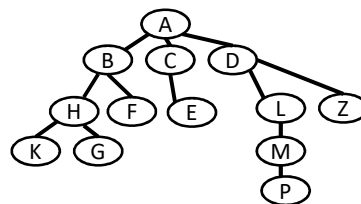
Se crea un registro de activación para inorden de L.

Como L tiene un hijo, que se llama M, se llama a inorden de M.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C

`Inorder(T) => in(A) => in(D) => in(L) => in(M)`

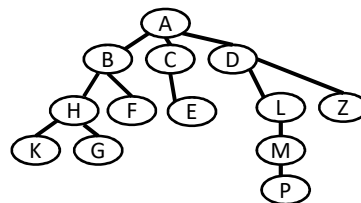
Se crea un registro de activación para inorden de M.

Como M tiene un hijo, que se llama P, se llama a inorden de P.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 Si `v` es hoja en `T` entonces
 `Visitar( T, v )`
 Sino
 `w` ← primer hijo de `v` en `T`
 `inorden( T, w )`
 `Visitar( T, v )`
 mientras `w` tiene hermano en `T` hacer
 `w` ← hermano de `w` en `T`
 `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C

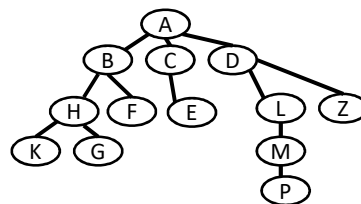
`Inorden(T) => in(A) => in(D) => in(L) => in(M) => in(P)`

*Se crea un registro de activación para inorden de P.
 Como P es una hoja, imprime P y sale.*

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 Si `v` es hoja en `T` entonces
 `Visitar( T, v )`
 Sino
 `w` ← primer hijo de `v` en `T`
 `inorden( T, w )`
 `Visitar( T, v )`
 mientras `w` tiene hermano en `T` hacer
 `w` ← hermano de `w` en `T`
 `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C P

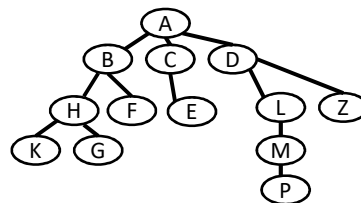
`Inorden(T) => in(A) => in(D) => in(L) => in(M)`

El control está de nuevo en M, imprime M y como no hay más hijos que explorar, sale.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C P M

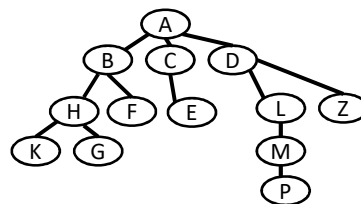
`Inorder(T) => in(A) => in(D) => in(L)`

El control está de nuevo en L, imprime L y como no hay más hijos que explorar, sale.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C P M L

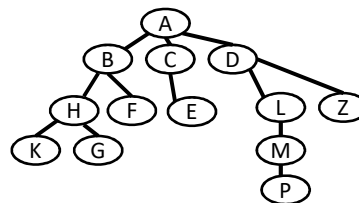
`Inorder(T) => in(A) => in(D)`

El control está de nuevo en D, imprime D y llama con el 2do hijo, que es Z.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C P M L D

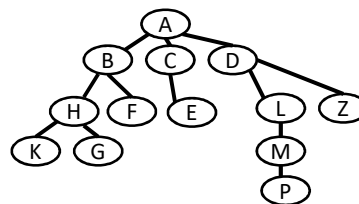
`Inorder(T) => in(A) => in(D) => in(Z)`

Como Z es una hoja, imprime Z y sale.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - Visitar(`T, v`)
 - Sino
 - `w` ← primer hijo de `v` en `T`
 - `inorden( T, w )`
 - Visitar(`T, v`)
 - mientras `w` tiene hermano en `T` hacer
 - `w` ← hermano de `w` en `T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C P M L D Z

`Inorder(T) => in(A) => in(D)`

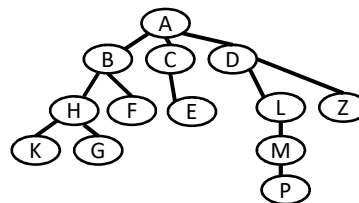
El control está en D.

Como no tiene más hijos, sale.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - `Visitar( T, v )`
 - Sino
 - `w ← primer hijo de v en T`
 - `inorden( T, w )`
 - `Visitar( T, v )`
 - mientras `w` tiene hermano en `T` hacer
 - `w ← hermano de w en T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C P M L D Z

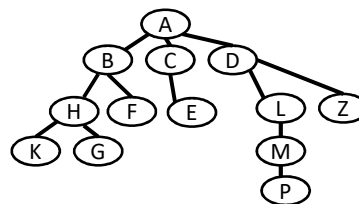
`Inorder(T) => in(A)`

*El control está en A de nuevo.
Como no tiene más hijos, sale.*

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- **Algoritmo** `inorden( T, v )`
 - Si `v` es hoja en `T` entonces
 - `Visitar( T, v )`
 - Sino
 - `w ← primer hijo de v en T`
 - `inorden( T, w )`
 - `Visitar( T, v )`
 - mientras `w` tiene hermano en `T` hacer
 - `w ← hermano de w en T`
 - `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C P M L D Z

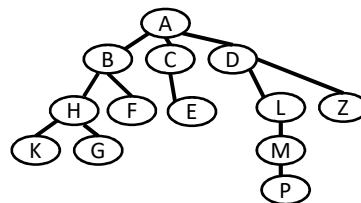
`Inorder(T)`

El control está en en el Shell nuevamente. Sale.

Recorridos de árboles: Inorden -- Traza

El algoritmo se llama con `inorden( T, T.root() )`

- Algoritmo `inorden( T, v )`
 Si `v` es hoja en `T` entonces
 `Visitar( T, v )`
 Sino
 `w` ← primer hijo de `v` en `T`
 `inorden( T, w )`
 `Visitar( T, v )`
 mientras `w` tiene hermano en `T` hacer
 `w` ← hermano de `w` en `T`
 `inorden( T, w )`



Salida esperada: K H G B F A E C P M L D Z

Salida: K H G B F A E C P M L D Z

La salida obtenida concuerda con la salida esperada.

Resumen de los recorridos

Planteo Preorden:

CB: $\text{pre}(\text{hoja}(e)) = [e]$

CR: $\text{pre}(\text{nodo}(e, [t_1 \ t_2 \ \dots \ t_k])) = [e] + \text{pre}(t_1) + \text{pre}(t_2) + \dots + \text{pre}(t_k)$

Planteo Postorden:

CB: $\text{post}(\text{hoja}(e)) = [e]$

CR: $\text{post}(\text{nodo}(e, [t_1 \ t_2 \ \dots \ t_k])) = \text{post}(t_1) + \text{post}(t_2) + \dots + \text{post}(t_k) + [e]$

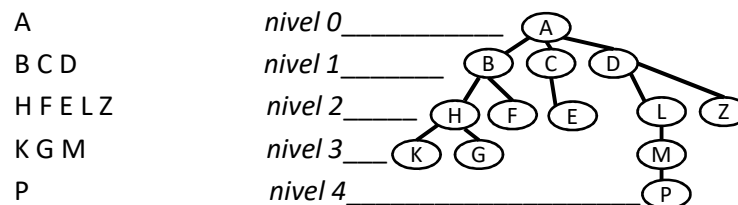
Planteo Inorden:

CB: $\text{in}(\text{hoja}(e)) = [e]$

CR: $\text{in}(\text{nodo}(e, [t_1 \ t_2 \ \dots \ t_k])) = \text{in}(t_1) + [e] + \text{in}(t_2) + \dots + \text{in}(t_k)$

Recorridos de árboles: por niveles

- Nivel: Subconjunto de nodos que tienen la misma profundidad
- Recorrido por niveles (level numbering): Visita todos los nodos con profundidad p antes de recorrer todos los nodos con profundidad $p+1$.
- Ejemplo: El recorrido/listado por niveles es:



Recorridos de árboles: por niveles (primera aproximación)

Algoritmo niveles(T)

Cola $\leftarrow$ new Cola()

Cola.enqueue(T.root())

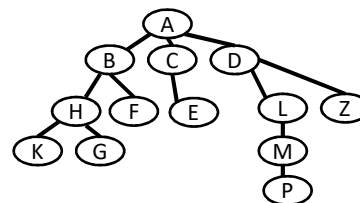
Mientras not cola.isEmpty()

$v \leftarrow$ cola.dequeue()

 mostrar v // visita de v

 para cada hijo w de v en T hacer

 cola.enqueue(w)



Genera el listado A B C D H F E L Z K G M P

Si n es la cantidad de nodos del árbol:

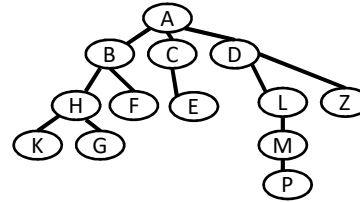
$$T_{\text{niveles}}(n) = O(n)$$

Recorridos de árboles: por niveles -- traza

```

Algoritmo niveles( T )
Cola ← new Cola()
Cola.enqueue( T.root() )
Mientras not cola.isEmpty()
    v ← cola.dequeue()
    mostrar v // visita de v
    para cada hijo w de v en T hacer
        cola.enqueue( w )

```



¿Genera el listado A B C D H F E L Z K G M P?

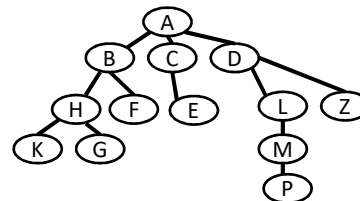
Crea una cola vacía, entonces cola = [].
Encola a A, entonces cola = [A].
Entra al while, como la cola no está vacía,
desencola a A y lo almacena en v, v=A, la cola quedó vacía, entonces cola = [].
Imprime v=A, entonces Salida = A
Encola a los hijos de A, entonces la cola = [B, C, D].

Recorridos de árboles: por niveles -- traza

```

Algoritmo niveles( T )
Cola ← new Cola()
Cola.enqueue( T.root() )
Mientras not cola.isEmpty()
    v ← cola.dequeue()
    mostrar v // visita de v
    para cada hijo w de v en T hacer
        cola.enqueue( w )

```



¿Genera el listado A B C D H F E L Z K G M P?

Salida = A y cola = [B, C, D].
Como la cola no está vacía, entra al while, desencola: v = B y cola = [C, D]
Muestra v, entonces Salida = A B.
Encola a los hijos de v=B en la cola, cola = [C, D, H, F].
Como la cola no está vacía, entra al while, desencola: v = C y cola = [D, H, F].
Muestra v=C, entonces Salida = A B C.
Encola a los hijos de v=C en la cola, cola = [D, H, F, E].

Recorridos de árboles: por niveles -- traza

Algoritmo niveles(T)

Cola ← new Cola()

Cola.enqueue(T.root())

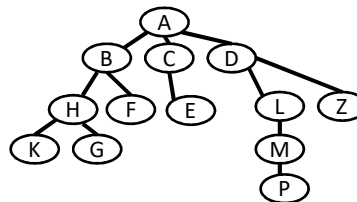
Mientras not cola.isEmpty()

v ← cola.dequeue()

mostrar v // visita de v

para cada hijo w de v en T hacer

cola.enqueue(w)



¿Genera el listado A B C D H F E L Z K G M P?

*Salida = A B C y cola = [D, H, F, E].**Como la cola no está vacía, entra al while, desencola: v = D y cola = [H, F, E].**Muestra v=D, entonces Salida = A B C D.**Encola a los hijos de v=D en la cola, cola = [H, F, E, L, Z].**Como la cola no está vacía, entra al while, desencola: v = H y cola = [F, E, L, Z].**Muestra v=H, entonces Salida = A B C D H.**Encola a los hijos de v=H, cola = [F, E, L, Z, K, G].**Entra al while de nuevo, desencola y v=F y cola = [E, L, Z, K, G].**Muestra v=F, Salida = A B C D H F.*

Estructuras de datos - Dr. Sergio A. Gómez

153

Recorridos de árboles: por niveles -- traza

Algoritmo niveles(T)

Cola ← new Cola()

Cola.enqueue(T.root())

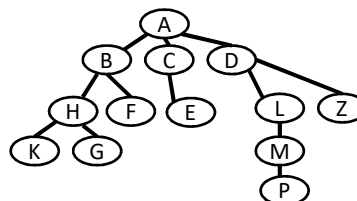
Mientras not cola.isEmpty()

v ← cola.dequeue()

mostrar v // visita de v

para cada hijo w de v en T hacer

cola.enqueue(w)



¿Genera el listado A B C D H F E L Z K G M P?

*v=F, Salida = A B C D H F y cola=[E, L, Z, K, G].**Como la cola no está vacía, entra al while, desencola: v = E y cola = [L, Z, K, G].**Muestra v=E, entonces Salida = A B C D H F E.**Encola a los hijos de v=E, cola = [L, Z, K, G].**Como la cola no está vacía, entra al while, desencola: v = L y cola = [Z, K, G].**Muestra v=L, entonces Salida = A B C D H F E L.**Encola a los hijos de v=L, cola = [Z, K, G, M].**Entra al while de nuevo, desencola y v=Z y cola = [K, G, M].*

Estructuras de datos - Dr. Sergio A. Gómez

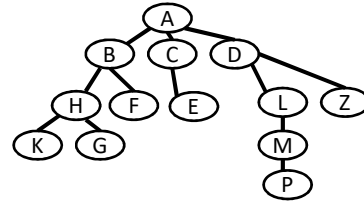
154

Recorridos de árboles: por niveles -- traza

```

Algoritmo niveles( T )
Cola ← new Cola()
Cola.enqueue( T.root() )
Mientras not cola.isEmpty()
    v ← cola.dequeue()
    mostrar v // visita de v
    para cada hijo w de v en T hacer
        cola.enqueue( w )

```



¿Genera el listado A B C D H F E L Z K G M P?

v=Z, Salida = A B C D H F E L y cola=[K, G, M].
Muestra v=Z, entonces Salida = A B C D H F E L Z. Como v=Z no tiene hijos, no encola nada.
Como la cola no está vacía, entra al while, desencola: v = K y cola = [G, M].
Muestra v=K, entonces Salida = A B C D H F E L Z K.
Como v=K no tiene hijos, no encola nada: cola = [G, M].
Como la cola no está vacía, entra al while, desencola: v = G y cola = [M].
Muestra v=G, entonces Salida = A B C D H F E L Z K G.
Como v=G no tiene hijos, no encola nada: cola=[M]
Entra al while de nuevo, desencola y v=M y cola = [].

Estructuras de datos - Dr. Sergio A. Gómez

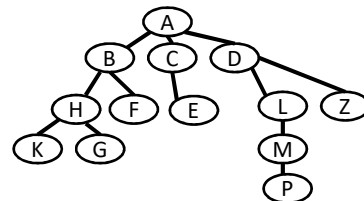
155

Recorridos de árboles: por niveles -- traza

```

Algoritmo niveles( T )
Cola ← new Cola()
Cola.enqueue( T.root() )
Mientras not cola.isEmpty()
    v ← cola.dequeue()
    mostrar v // visita de v
    para cada hijo w de v en T hacer
        cola.enqueue( w )

```



¿Genera el listado A B C D H F E L Z K G M P?

v=M, Salida = A B C D H F E L Z K G y cola=[].
Muestra v=M, Salida = A B C D H F E L Z K G M.
Encola a los hijos de v=M, cola = [P].
Entra al while porque la cola no está vacía, desencola, v=P y cola=[].
Muestra v=P, Salida = A B C D H F E L Z K G M P.
Como v=P no tiene hijos, no encola nada: cola = [].
Como cola está vacía termina.
Vemos que como Salida = A B C D H F E L Z K G M P, la traza es exitosa.

Estructuras de datos - Dr. Sergio A. Gómez

156

Tiempo de ejecución de listado por niveles

```

Algoritmo niveles( T )
Cola ← new Cola() cte
Cola.enqueue( T.root() )
Mientras not cola.isEmpty() n vueltas, condición cte
    v ← cola.dequeue() cte
    mostrar v cte
    para cada hijo w de v en T hacer hi vueltas
        cola.enqueue( w ) cte

```

- Entrada: árbol T
- Tamaño de la entrada: n = cantidad de nodos del árbol T
- Tiempo de ejecución:

Sea h_i la cantidad de hijos del nodo i

$$T(n) = c_1 + \sum_{i=1}^n (c_2 + c_3 h_i) =$$

$$= c_1 + c_2 n + c_3 (n-1) = O(n)$$

Recordar que la suma de la cantidad de hijos de todos los nodos es n-1, porque es igual a la cantidad de padres en el árbol y todos los nodos tienen exactamente un padre menos la raíz que no tiene.

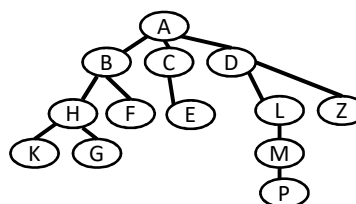
Recorridos por niveles mostrando dónde termina cada nivel

```

Algoritmo niveles( T )
Cola ← new Cola()
Cola.enqueue( T.root() )
Cola.enqueue( null ) // Esto me sirve para detectar dónde va un fin de línea
Mientras not cola.isEmpty()
    v ← cola.dequeue()
    si v ≠ null entonces
        mostrar v // visita de v
        para cada hijo w de v en T hacer
            cola.enqueue( w )
    sino
        imprimir fin de línea
        si not cola.isEmpty() entonces
            cola.enqueue( null )

```

Nota: $T_{\text{niveles}}(n) = O(n)$ Al tener en cuenta los null, se agregan a lo sumo n iteraciones, con lo que no cambia el orden.

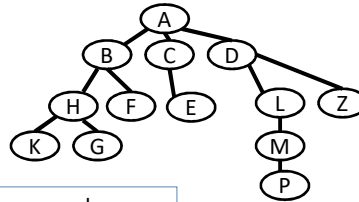


A
B C D
H F E L Z
K G M
P

Recorridos por niveles mostrando dónde termina cada nivel

Algoritmo niveles(T)

```
Cola ← new Cola()
Cola.enqueue( T.root() )
Cola.enqueue( null )
Mientras not cola.isEmpty()
    v ← cola.dequeue()
    si v ≠ null entonces
        mostrar v
        para cada hijo w de v en T hacer
            cola.enqueue( w )
    sino
        imprimir fin de línea
        si not cola.isEmpty() entonces
            cola.enqueue( null )
```



Salida esperada:

```
A
B C D
H F E L Z
K G M
P
```

Salida es vacía.

Cola = [A, null]. Como la cola no está vacía entra al while.

Desencola y v=A y cola=[null].

Como v=A!=null muestra A, salida=A y encola los hijos de A. Cola=[null, B, C, D].

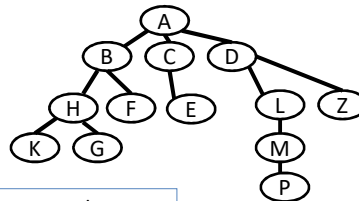
Estructuras de datos - Dr. Sergio A. Gómez

159

Recorridos por niveles mostrando dónde termina cada nivel

Algoritmo niveles(T)

```
Cola ← new Cola()
Cola.enqueue( T.root() )
Cola.enqueue( null )
Mientras not cola.isEmpty()
    v ← cola.dequeue()
    si v ≠ null entonces
        mostrar v
        para cada hijo w de v en T hacer
            cola.enqueue( w )
    sino
        imprimir fin de línea
        si not cola.isEmpty() entonces
            cola.enqueue( null )
```



Salida esperada:

```
A
B C D
H F E L Z
K G M
P
```

Cola=[null, B, C, D]. Entra al while y desencola v=null, entonces cola = [B, C, D].

Como v=null, imprime EOL a la salida y Salida = A EOL.

Como la cola no está vacía vuelve a encolar null.

Cola=[B,C,D,null]... continuarlo y ver que al final de la ejecución, al sacar el ultimo null, la cola queda vacía y termina.

Estructuras de datos - Dr. Sergio A. Gómez

160

Búsqueda en un árbol:
Recorrido no exhaustivo

Buscar(t, x): buscar_rec(t, t.root(), x)
 Buscar recursivo se define por casos: buscar_rec(t, p, x)
CB: rotulo(p) = x => buscar_rec(t, p, x) = p
CB: rotulo(p) ≠ x y p es hoja => buscar_rec(t, p, x) = null
CR: rotulo(p) ≠ x y p es interno =>
 sean hi los hijos de p:
 buscar_rec(t, p, x)
 = buscar_rec(t, hi, x),
 si $\exists hi : \text{buscar_rec}(t, hi, x) \neq$
 = null,
 si $\forall hi. \text{buscar_rec}(t, hi, x) = \text{null}$

Estructuras de datos - Dr. Sergio A. Gómez

161

Búsqueda en un árbol:
Recorrido no exhaustivo

Buscar(t, x): buscar_rec(t, t.root(), x)
Algoritmo buscar_rec(t, p, x):
 si rotulo(p) = x:
 retornar p
 sino:
 si p es hoja: retornar null
 sino:
 para cada hijo h de p:
 res = buscar_rec(t, h, x)
 si res ≠ null: retornar res
 retornar null

Estructuras de datos - Dr. Sergio A. Gómez

162

Bibliografía

- Goodrich & Tamassia, Data Structures and Algorithms in Java, 4th Edition, capítulo 7.